# Незалежна перевірка локального компілятора Фур'є з препринту OpenAI «An explicit power saving for the exact discrete Fourier transform»: точні й чисельні тести, глибина, вартість і межі застосовності

Олександр Пилипенко
ORCID: 0009-0004-9610-9242
op1@cyberunit.tech, limblucifer666@gmail.com

2026-10-14

**Анотація**

Ми незалежно перевіряємо одну складову препринту OpenAI «An explicit power saving for the exact discrete Fourier transform», який стверджує існування точного детермінованого алгоритму DFT для кожної довжини $n$ за час $O\big(n(\log n)^{\theta}(\log\log n)^{4-\theta}\big)$, $\theta = 1 - 2{,}1 \cdot 10^{-13}$, з точною комплексною арифметикою і необмеженими коефіцієнтами. Показник дає тензорна мережа, яку ми не реалізовували. Ми реалізували локальний компілятор Фур'є (ньютонову редукцію $F_r(u)$ до трикутних Тепліцевих множників, блочне Тепліцеве слово точної ширини з реплеєм згорткових DAG і шаблон зсувів з $C$-слотами), чия межа $O(\log^4 r)$ слотів дає множник $(\log\log n)^4$, а не $\theta$. Усі результати стосуються строгої конструкції, топологія якої залежить лише від ширини. Обчислення в $\mathbb{Q}(\zeta_N)$ доводить, що реалізована матриця дорівнює $F_r(u)$ при $r = 4, 7, 8, 9, 12, 15, 16, 32$ (шлях через $C$-слоти — при $r = 4, 8, 16$). Точні образи в $\mathbb{F}_p$ збігаються на вибраних ширинах до 512, а в потоковому тесті — і при $r = 1024$; це лише необхідна умова. Дві точності mpmath, що відрізняються на 100 знаків, зменшують похибку приблизно на 100 порядків на всіх ширинах 7–256. Хоча $\kappa_2(F_r) = 1$, слово числово нестійке: $\log_{10}\kappa_2(D_H)$ має головний член $cr$, $c = \mathrm{Cl}_2(\pi/3)/(\pi \ln 10) \approx 0{,}1403$, ланцюжок втрачає 142 знаки при $r = 256$, а у float64 відносна похибка при $r = 64$ сягає $8 \cdot 10^{23}$; препринт не робить тверджень про скінченну точність. Хеші структури збігаються для трьох входів на 12 ширинах 64–1024, а модель глибини точно збігається з виміряною ASAP-глибиною на 10 із них. Ми також перекомпілювали Lean-формалізацію OpenAI і провели аудит її формулювання.

**Ключові слова:** дискретне перетворення Фур'є; схемна складність; глибина схеми; Тепліцева матриця; числова стійкість; циклотомічні поля; точна арифметика; формальна верифікація (Lean).

**Заява про незалежність.** Це незалежна перевірка частини препринту OpenAI. Станом на 2026-10-14 контакту з OpenAI щодо цієї роботи не було; перевірку не узгоджували з авторами препринту, і вони її не рецензували. Нового алгоритму DFT не запропоновано.

## 1 Вступ

### 1.1 Мотивація

Препринт OpenAI [9] стверджує, що DFT довжини $n$ можна обчислити точно за час $o(n \log n)$ у моделі з одиничною вартістю арифметичних операцій над комплексними числами і одиничною вартістю цілочисельних операцій над словами з $O(\log n)$ біт. Алгоритм

Кулі–Тьюкі [5] дає $O(n \log n)$. Відомі нижні межі для суміжних моделей [1, 2, 8] не переносяться на цю модель автоматично (підрозділ 2.5). Твердження незвичне. Сам препринт застерігає, що сталі алгоритму величезні, що результат асимптотичний, «with no useful crossover estimate», і що висновок не стосується стійких FFT з плаваючою комою чи бітової складності (`introduction.tex:101`). Схожі практичні зауваження робить Recht [11] (приховані сталі; точна арифметика над числами, що ростуть). Тому варто незалежно перевірити хоча б ті частини доведення, які можна перевірити обчисленням, і виміряти, як у них проявляються обмеження моделі, що їх препринт називає сам.

**Джерело та авторство.** Препринт не є окремою статтею з DOI. Це текст у репозиторії `https://github.com/openai/math`, каталог `preprints/An-explicit-power-saving-for-the-exact-discrete-Fourier-transform-September-25-2026`: головний файл `main.pdf`, вихідні файли `build/main.tex`, розділи `build/sections/*.tex` і бібліографія `build/references.bib`. Ми працювали зі станом каталогу на коміті 301488868 (2026-10-07; HEAD репозиторію fd4aeeb) і посилаємося на твердження за їхніми мітками (`thm:main`, `cor:decimal`, `loc:compiler`, ...), а на рядки джерел — у вигляді `file.tex:line`. В авторському блоці і в README каталогу автором зазначено «OpenAI» (`build/main.tex`, `\author{OpenAI}`; `README.md`, «Author: OpenAI»); іменних авторів немає. За README репозиторію, рукописи колекції створено внутрішньою моделлю OpenAI. Lean-формалізація, яку ми перекомпілювали (підрозділ 4.8), — окремий артефакт того самого репозиторію (каталог `lean/`, опис обсягу — `lean/docs/130.md`), а не частина тексту препринту.

### 1.2 Що перевіряється

Ми перевіряємо локальний компілятор з `local.tex`: правильність реалізованого оператора (точно і в арифметиці високої точності), його числову поведінку, глибину за ASAP-розкладом і вартість. Наші локальні експерименти не відтворюють головної теореми й не перевіряють асимптотичного виграшу з показником $\theta$; окремо ми відтворили механічну перевірку Lean-цілі (підрозділ 4.8). У таблиці 1 підсумовано статус кожної складової препринту.

Сила доказу різна, і ми розрізняємо її рівні (підрозділ 4.1). За спаданням: (1) *доведення обчисленням*, тобто точна рівність у $\mathbb{Q}(\zeta_N)$ для восьми ширин 4–32 (повний ланцюжок) і для $r = 4, 8, 16$ ($C$-слоти); (2) *точні, але лише необхідні умови*, тобто рівність образів у $\mathbb{F}_p$ для вибраних ширин до 512; (3) *числова узгодженість*, тобто mpmath з двома точностями до $r = 256$; (4) *вимірювання конкретного слова*: топологія, ASAP-глибина, гейти, флопи, час. Локальними експериментами не перевірено: мережу, асимптотичний виграш із $\theta$, синхронізацію, довжини всіх розмірів, синтез. Наші результати не підтверджують і не спростовують головної теореми.

### 1.3 Що не входить у роботу

Ми не реалізовували і не перевіряли:

- мережу з `network.tex` (бінарні рамки, бюджет вимірів, тензорний степінь ядра $C$ і лему `net:tensor-bound`, з якої походить $\theta$);
- синхронізацію тензорних множників (`synchronization.tex`);
- довжини всіх розмірів через китайську теорему про залишки і чирп-згортку (`all-lengths.tex`);
- синтез і принтер (`synthesis.tex`).

**Таблиця 1:** Статус складових препринту в цьому дослідженні.

| Складова (файл / мітка препринту) | Що зроблено тут | Статус |
|---|---|---|
| Ньютонова редукція (`loc:newton-reduction`) | Повний ланцюжок DFT: точно в $\mathbb{Q}(\zeta_N)$ і $\mathbb{F}_p$; у mp з двома точностями | Доведення обчисленням для всієї матриці при $r = 4, 7, 8, 9, 12, 15, 16, 32$ ($\mathbb{Q}(\zeta_N)$); необхідна умова на вибраних $r \le 128$ ($\mathbb{F}_p$) |
| Тепліцеве слово (`loc:toeplitz`), строгий режим | Глибина, гейти, топологія (аудит), точність $Tx$ | Виміряно на вибраних ширинах до $r = 1024$; точність і обсяг — таблиці 2–11 |
| Реплей DAG (`loc:replay`) | Виконання за раундами дорівнює послідовному; конфліктів немає | Перевірено (строгий режим) |
| Шаблон $C$-слотів (`loc:compiler`) | Власний замкнений розклад зсуву; розщеплення без тесту на нуль; спряження через DAG з оберненими коренями | Доведення обчисленням при $r = 4, 8, 16$ ($\mathbb{Q}(\zeta_N)$); необхідна умова на вибраних $r \le 128$ ($\mathbb{F}_p$); mp до $r = 256$ |
| Правило розміру чанка | Еквівалентне правилу препринту для $b \le 1024$ (підрозділ 3.3) | Перевірено для $b \le 1024$ |
| Конкретна мережа з `network.tex` та її асимптотичний виграш | Не реалізовано; числове $\theta$ перераховано (підрозділ 2.2); Lean для іншої мережі — у підрозділі 4.8 | Локальними експериментами не перевірено |
| Синхронізація (`synchronization.tex`) | Не реалізовано | Не перевірено |
| Довжини всіх розмірів (`all-lengths.tex`) | Не реалізовано | Не перевірено |
| Синтез і принтер (`synthesis.tex`) | Не реалізовано | Не перевірено |
| Lean-формалізація (`lean/OAI/Computability/FourierTransform`) | Скомпільовано: 0 помилок, 0 попереджень про `sorry`; аксіоми лише `propext`, `Classical.choice`, `Quot.sound` | Механічно перевірено; звірку означень завершено в заявленому обсязі; адекватність `Bill` і конкретна `network.tex` — поза аудитом |

Показник $\theta$ дає мережа, а не локальний компілятор. Локальний компілятор — необхідна складова доведення (його $O(\log^4 r)$ слотів дають множник $(\log\log n)^4$ головної межі; `synchronization.tex:74`, `all-lengths.tex:137`), але не джерело економії. Наші результати стосуються цієї складової, і переносити їх на головну теорему не можна.

### 1.4 Внесок

Ми не пропонуємо нового алгоритму DFT. Наш внесок — незалежна реалізація і перевірка локального компілятора Фур'є на вибраних ширинах, кількісне дослідження його числової нестійкості та відтворювані інструменти точної перевірки, зокрема потокове обчислення великих обчислювальних графів. Конкретно, дослідження дає:

(а) Відтворювану реалізацію і перевірку повного ланцюжка ньютонової редукції і Тепліцевого слова для DFT: точно в циклотомічних і скінченних полях та в арифметиці високої точності, з окремими тестами загального Тепліцевого оператора і шляху через $C$-слоти. Рівні доказовості і перевірені ширини наведено в підрозділі 4.1.

(б) Незалежно виведений алгебраїчний розклад зсуву через $C$-слоти; матричну тотожність і умови ненульовості параметрів доведено в підрозділі 3.5.

(в) Аналітичний опис обумовленості $D_H$ у випадку Фур'є: точні скінченні часткові суми і головний асимптотичний член $cr$ для $\log_{10}\kappa_2(D_H)$ через функцію Клаузена (додаток Б.2).

(г) Кількісне дослідження втрати знаків по етапах, росту проміжних величин по рівнях рекурсії і фазах та чутливості похибки до точності фаз для перевірених входів (підрозділ 4.5). Ці діагностики не дають повного причинного розкладу похибки округлення.

(д) Вимірювання глибини, роботи, часу і пам'яті строгої конструкції з перевіреною фіксованою топологією; порівняння з FFT і прямим множенням в однакових одиницях вартості; виявлення артефактів глибини, спричинених залежним від значень пропуском нульових множників у нашій реалізації (підрозділ 3.6).

(е) Відтворюваний потоковий точний перевірник у скінченних полях, що обчислює пряме і транспоноване слова без матеріалізації всього графа, зокрема окрему перевірку при $r = 1024$ на тих самих двох векторах у двох полях. Її обсяг залишається точним тестом необхідної умови, а не доведенням для всієї комплексної матриці.

Цей внесок стосується незалежно розробленої реалізації, аналізу та інструментів перевірки; пріоритету щодо математичних тотожностей чи асимптотик обумовленості ми не заявляємо. Виміряні обмеження (підрозділи 5.2 і 5.6) стосуються скінченної точності або окремого виконання локального слова — режимів, про які препринт тверджень не робить. Препринту вони не спростовують.

# 2 Твердження препринту

Цей розділ іде за текстом препринту, а не за нашим його прочитанням.

## 2.1 Модель обчислень

За розділом «Arithmetic model and main result» препринту [9]: є комплексні регістри і цілочисельні адресні регістри. Одна операція поля коштує одиницю. Цілочисельні операції та довільний доступ до пам'яті на словах з $O(\log(n+2))$ біт коштують одиницю. Дозволені додавання, віднімання і множення на підготовлені скаляри, які не залежать від вхідних даних. Ділення дозволене лише при підготовці скалярів, і знаменник має бути доведено ненульовим. Один явно заданий корінь з одиниці $\zeta_D$, $D = D_* < 1024\,n^3$, подається з одиничною вартістю. Обмежень на модуль коефіцієнтів немає, а стійкості обчислень з плаваючою комою препринт не розглядає і не заявляє: «There is no bound on complex coefficients or intermediate magnitudes, and no finite-precision or numerical-stability assertion» (`introduction.tex:67`). Там само препринт застерігає, що сталі величезні, що корисної оцінки точки перетину немає і що висновок «does not assert a corresponding improvement for stable floating-point FFTs or for bit complexity» (`introduction.tex:101`).

## 2.2 Сталі і показник

Препринт означує

$$m = 10^6, \qquad W_* = 2^{71}, \qquad \Delta = 6871402692000000, \qquad \lambda = m - \frac{\Delta}{W_*}, \qquad \theta = \log_m \lambda, \quad (1)$$

тож $\theta = 0{,}99999999999978935615699598\ldots < 1$. Позначимо

$$\delta = 1 - \theta \approx 2{,}106 \cdot 10^{-13}. \quad (2)$$

Ми незалежно перерахували $\theta$ (mpmath, 50 знаків) і отримали те саме значення.

## 2.3 Головна теорема і десятковий наслідок

**Теорема 1** ([9, Theorem `thm:main`]). *Існує один детермінований точний алгоритм, який для кожного $n$ і кожного $x \in \mathbb{C}^n$ обчислює $F_n x$ за час*

$$O\big(n(\log n)^{\theta}(\log\log n)^{4-\theta}\big) \qquad (n \to \infty). \tag{3}$$

*Час включає підготовку скалярів, побудову розкладу і роботу з індексами.*

**Наслідок 1** ([9, Corollary `cor:decimal`]). *У тій самій моделі перетворення обчислюється за час $O\big(n(\log n)^{1-10^{-13}}\big)$, тобто $o(n \log n)$.*

## 2.4 Місце локального компілятора

Головну теорему отримано з мережі: `net:tensor-bound` застосовує $C^{\otimes k}$ до масиву довжини $2^k$ за час $O\big(2^k(k+1)^{\theta}\big)$. Локальний компілятор (`loc:compiler`) і Тепліцеве слово (`loc:toeplitz`) не дають $\theta$, але є необхідною частиною конструкції: слова для всіх осей доповнюються до $O\big((1+\log r_{\max})^4\big)$ слотів, кожен слот виконує мережа, і звідси множник $(\log \ell)^4$ у

$$T(n) = O\big(n\,\ell^{\theta}(\log \ell)^4\big), \qquad \ell = \Theta\Big(\frac{\log n}{\log\log n}\Big) \tag{4}$$

(`synchronization.tex:74--82`; `all-lengths.tex:137`). Вартість самого локального слова в гейтах у межу не входить; входять лише кількість слотів і поліноміальна підготовка. Локальні ширини — перші $\ell$ непарних простих і один степінь двійки (`all-lengths.tex:13--56`). Препринт зазначає, що механізм Фур'є точної ширини спільний із розділом 3 супровідного препринту *Finite tensor savings and exact Fourier circuits* [10] (`introduction.tex:158`, `local.tex:10--11`).

**Що стверджує `local.tex`.** Для первісного кореня $r$-го степеня з одиниці $u$ покладемо $F_r(u) = (u^{ij})_{0 \le i,j < r}$.

- *Лема `loc:newton-reduction`.* $F_r(u) = N D_N^{-1} N^{\mathsf{T}}$, $D_N = \operatorname{diag}(N_{00}, \dots, N_{r-1,r-1})$, де $N$ — оборотна нижня трикутна матриця, що є добутком двох діагональних матриць і однієї нижньої трикутної Тепліцевої. За формулою `loc:newton`

$$N_{ik} = (-1)^k u^{k(k-1)/2}\,\frac{H_i}{H_{i-k}}, \qquad H_j = \prod_{s=1}^{j}(1-u^s), \quad H_0 = 1, \tag{5}$$

  тобто $N = D_H\, T\, D_c$ з $D_c = \operatorname{diag}(c_k)$, $c_k = (-1)^k u^{k(k-1)/2}$, і $D_N = \operatorname{diag}(c_k H_k)$.

- *Лема `loc:toeplitz`.* Оборотна нижня трикутна Тепліцева матриця ширини $r$ має слово на рівно $r$ координатах з $O(1+\log^4 r)$ змішаними раундами; «the topology depends only on $r$». Це верхня межа; оцінки $\Theta(\log^4 r)$ препринт тут не стверджує.

- *Правило розміру чанка* (там само). Для кожного $b \in \{1, \dots, v\}$ інтервали розбиваються на чанки розміру $b$ (крім, можливо, останнього); для кожної пари друкується скелет DAG і рахуються його гейти $g$; $b$ допустиме, якщо $g + a + e \le v$ для кожної пари, де $a, e$ — розміри цільового і джерельного чанків; обирається найбільше допустиме $b$. Якщо допустимого немає, використовується пряме слово.

- *Рекурентність глибини* в доведенні:

$$d(v) \le \max\big\{d(\lfloor v/2 \rfloor),\, d(\lceil v/2 \rceil)\big\} + O(1+\log^3 v). \tag{6}$$

- *Шаблон слотів* (`loc:compiler`). Кожен верхній зсув з ненульовим коефіцієнтом $t$ має слово з трьома прямими копіями $C$ і діагональними множниками (формули `loc:three-kernel` і `loc:nonzero-shear`): з $H' = \left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$

$$\begin{aligned} K &= H'\,\mathrm{diag}(2,1)H'\,\mathrm{diag}(-3,1)H' = \begin{pmatrix} -8 & -10 \\ 0 & -6 \end{pmatrix}, \\ \begin{pmatrix} 1 & t \\ 0 & 1 \end{pmatrix} &= D_t\,\mathrm{diag}(-1/8,-1/6)\,K\,D_t^{-1}, \qquad D_t = \mathrm{diag}(4t/5,1). \end{aligned} \tag{7}$$

  Довільний коефіцієнт $\mu$ розщеплюється як

$$\kappa = 1 + \mu\bar{\mu}, \qquad t_1 = \kappa, \qquad t_2 = \mu - \kappa, \tag{8}$$

  обидва ненульові, що дає однаковий шаблон із шести $C$-слотів для кожного коефіцієнта. Спряжене $\bar{\mu}$ є підготовленим скаляром: той самий арифметичний DAG обчислюється вдруге з оберненими коренями.

### 2.5 Нижні межі, згадані в препринті

Morgenstern [8] дає нижню межу для ненормованого перетворення з обмеженими коефіцієнтами. Ailon [1, 2] дає межі для нормованого перетворення з унітарними гейтами $2 \times 2$ і для моделі з обмеженою обумовленістю. Модель препринту допускає необмежені коефіцієнти, тож ці межі не переносяться на неї автоматично. Це пояснює відсутність прямої суперечності, але саме по собі не підтверджує доведення.

### 2.6 Пов'язані роботи

Recht [11] (2026-10-12) називає результат «галактичним»: приховані сталі роблять його нерелевантним на фізично досяжних розмірах, а модель вимагає множити довільно великі комплексні числа за одну операцію. Обидва обмеження препринт називає сам (`introduction.tex:67`, `:101`), тож ця критика стосується практичної цінності, а не правильності. Наші дані (підрозділи 5.2, 5.6 і 6.1) кількісно описують саме ці обмеження для локальної складової, але не стосуються мережі і не встановлюють нижніх меж вартості будь-якої майбутньої модифікації. Displacement-конструкцію чанків препринт бере з роботи Kailath, Kung і Morf [7]. Нестійкість швидких методів для Тепліцевих систем відома давно (Bunch [4]); наш випадок інший (множення, а не розв'язання), але механізм схожий: великі проміжні величини. Обумовленість ньютонової форми залежить від порядку вузлів: природний порядок $u^0, u^1, \ldots$ вузлів на колі дає експоненційний ріст, а порядок Леджа його пригнічує (Reichel [12]). Наша формула для $\kappa(D_H)$ (підрозділ 5.2) кількісно описує саме природний порядок, який використовує препринт. Чи можна змінити порядок вузлів, не порушивши симетрії ньютонової редукції, ми не досліджували. Пряму і зворотну похибки використовуємо в сенсі Higham [6].

## 3 Реалізація локального компілятора

### 3.1 Ньютонова редукція

Для первісного кореня $u$ порядку $r$ (у тестах DFT $u = e^{2\pi i/r}$) нехай $H_j$ такі, як у (5), $c_k = (-1)^k u^{k(k-1)/2}$ і $T_{ij} = 1/H_{i-j}$ при $i \geq j$. Ми застосовуємо

$$F_r(u) = D_H\,T\,D_{\mathrm{mid}}\,T^{\mathsf{T}}D_H, \qquad D_{\mathrm{mid}} = \mathrm{diag}\big(c_k/H_k\big). \tag{9}$$

Це тотожність препринту $F_r(u) = ND_N^{-1}N^{\mathsf{T}}$ з $N = D_H T D_c$ і $D_N = \mathrm{diag}(c_k H_k)$, бо центральний множник дорівнює $D_c D_N^{-1} D_c = \mathrm{diag}(c_k^2/(c_k H_k)) = \mathrm{diag}(c_k/H_k)$. Знаменники ненульові, бо $u^s \neq 1$ при $1 \le s < r$. Тут потрібне звичайне транспонування $T^{\mathsf{T}}$, а не ермітове спряження. Слово для $T^{\mathsf{T}}$ отримується зі слова для $T$ оберненням порядку шарів і перестановкою кінців кожного зсуву (`compiler.transpose_word`). Матриці ширини $r$ потрібні лише $h_0, \ldots, h_{r-1}$, $h_k = 1/H_k$; оскільки $H_r = 0$, $h_r$ не визначене. Наш mp-код обчислює $h_r$, але ніде його не використовує; у точних перевірках ми покладаємо $h_r = 0$, і рівність виконується.

### 3.2 Блочне слово

Блок ширини $v$ ділиться на половини розмірів $\lfloor v/2 \rfloor$ і $\lceil v/2 \rceil$, які обробляються рекурсивно на непересічних координатах. Потім міжблокова частина обробляється чанками розміру $b$: для кожної пари (цільовий чанк $I$ з другої половини, джерельний чанк $J$ з першої) будується згортковий DAG із трьох зовнішніх добутків (displacement-конструкція [7]), який виконується реплеєм з координатами, позиченими з того самого блоку. Якщо допустимого $b$ немає, блок обробляється прямим словом.

### 3.3 Правило розміру чанка: наше і препринту

Наша функція `rounds.admissible_chunk` рахує гейти лише для повної пари $b \times b$ і зупиняється на першому $b$, для якого $g(b) + 2b > v$. Препринт перебирає всі $b$ і перевіряє всі пари, включно з неповними останніми чанками. На перевіреному діапазоні обидва правила дають одне й те саме $b$ з двох причин.

- Кожна фактична пара, включно з неповними, проходить перевірку $g + a + e \le v$: це assert у `block_word_local` («pair does not fit its block»), і він не спрацював у жодному вимірюванні адаптивного правила.
- Функція $b \mapsto g(b) + 2b$ не спадає при $1 \le b \le 1024$ (перевірено за повними таблицями $g(b)$ в обох режимах). Отже, допустимого $b$, більшого за перше недопустиме мінус один, немає. При $b > \lfloor v/2 \rfloor$ повних пар немає, а пара з цілих половин не вміщується, бо $g > 0$.

Поза $b \le 1024$ монотонності ми не доводили.

### 3.4 Закони лічби

Наші вимірювання дають такі закони; $g(b)$ — кількість гейтів, а $L(b)$ — кількість шарів реплею однієї пари $b \times b$.

- Строгий режим: $g(1) = 8$; режим з пропуском нулів: $g(1) = 6$. Строга таблиця $g(b)$ відрізняється від таблиці з пропуском нулів лише при $b = 1$; отже, в обох режимах

$$g(b) = b\,(24 \log_2 b + 38) \qquad \text{для степенів двійки } 2 \le b \le 1024. \tag{10}$$

- $L(1) = 28$ у строгому режимі (18 з пропуском нулів), $L(2) = 94$ і

$$L(b) = 62 + 32 \log_2 b \qquad \text{для степенів двійки } 4 \le b \le 128 \tag{11}$$

  (перевірено для $b = 4, 8, \ldots, 128$ в обох режимах; у режимі з пропуском нулів також для $b = 256$ і 1024).

- Наслідок $g(1) = 8$: у строгому режимі чанк розміру 1 вміщується лише при $v \ge 10$ (а не $v \ge 8$). Правило «чанк 1» у строгих вимірюваннях використовує саме цю умову.

### 3.5 Розклад зсуву через $C$-слоти

Ми використовуємо власний замкнений розклад нижнього зсуву. Нехай $C = \left(\begin{smallmatrix} p & q \\ q & p \end{smallmatrix}\right)$, $p = (1+i)/2$, $q = (1-i)/2$, і

$$D_0 = \mathrm{diag}\Big(-\frac{5i}{3c},\, 1\Big), \quad D_1 = \mathrm{diag}(2, 1), \quad D_2 = \mathrm{diag}(3, -1), \quad D_3 = \mathrm{diag}\Big(\frac{2c}{5(1+i)},\, \frac{1}{2(1-i)}\Big). \tag{12}$$

**Лема 1.** *Для кожного $c \neq 0$*

$$\begin{pmatrix} 1 & 0 \\ c & 1 \end{pmatrix} = D_0\, C\, D_1\, C\, D_2\, C\, D_3. \tag{13}$$

*Доведення.* Пряме множення дає

$$C\, D_1\, C\, D_2\, C = \begin{pmatrix} \frac{3}{2}(-1+i) & 0 \\ \frac{5}{2}(1+i) & 2(1-i) \end{pmatrix}. \tag{14}$$

Після множення на $D_0$ зліва і на $D_3$ справа елемент $(1,1)$ стає $-\frac{5i}{3c} \cdot \frac{3}{2}(-1+i) \cdot \frac{2c}{5(1+i)} = \frac{-i(-1+i)}{1+i} = 1$, елемент $(2,2)$ стає $2(1-i)\cdot\frac{1}{2(1-i)} = 1$, а елемент $(2,1)$ стає $\frac{5}{2}(1+i)\cdot\frac{2c}{5(1+i)} = c$. $\square$

Це алгебраїчна тотожність із трьома копіями $C$. Препринт використовує інший розклад того самого вигляду (верхній зсув через $H'$ і $K$, (7), також із трьома копіями $C$).

Для довільного коефіцієнта ми, як і препринт, розщеплюємо $\mu = t_1 + t_2$ згідно з (8).

**Лема 2.** *Для кожного $\mu \in \mathbb{C}$ з $\kappa = 1 + |\mu|^2$ маємо $t_1 = \kappa \geq 1$ і*

$$|t_2| = |\mu - \kappa| \;\geq\; \kappa - |\mu| = \Big(|\mu| - \tfrac{1}{2}\Big)^2 + \tfrac{3}{4} > 0. \tag{15}$$

Доведення наведено в додатку Б.1. Це дає шість $C$-слотів на кожен зсув без жодного тесту на нуль.

**Спряження.** Як і в препринті, спряжене $\bar{\mu}$ готується повторною побудовою всього слова з оберненими коренями ($u \mapsto u^{-1}$, кожен корінь порядку $L$ замінено оберненим; `mp_exact.ROOT_SIGN = -1`). Оскільки раціональні сталі дійсні, коефіцієнти другого слова дорівнюють $\bar{\mu}$. Ми перевірили, що структура другого слова збігається з першим, а відносна різниця з $\mathrm{conj}(\mu)$ не перевищує $10^{-59}$ ($r = 16$, 60 знаків); у точних полях вона нульова. У $\mathbb{Q}(\zeta_N)$ спряження — це автоморфізм $\zeta \mapsto \zeta^{-1}$. Просте поле $\mathbb{F}_p$ не має нетривіальних автоморфізмів; заміна коренів оберненими там просто редукує той самий комплексно спряжений вираз (образ $\zeta \mapsto \zeta^{-1}$ циклотомічного поля). Рядки $C$-слотів таблиці 2 і вся таблиця 4 використовують саме цей варіант.

### 3.6 Строга конструкція і статичний аудит розгалужень

**Строга конструкція.** Усі результати цієї статті використовують строгу конструкцію, у якій жоден гейт не відкидається через нульовий коефіцієнт (`conv_dag.KEEP_ZERO_MULTIPLIERS = True`). Саме це приписує препринт: топологію згортки «can be printed before any scalar coefficients are evaluated, retaining even gates with a coefficient that might happen to vanish» (`local.tex:140--142`); усі три зовнішні добутки зберігаються «even when some coefficients vanish» (`local.tex:268--269`); і «every structural choice uses integers alone» (`local.tex:42`).

Варіант, що пропускає точковий множник, коли його коефіцієнт дорівнює нулю точно (умова `abs(coef) == 0` у `conv_dag.py`), не змінює оператора, бо пропущений множник нульовий. Але він залежить від значень: йому потрібен тест на нуль комплексного коефіцієнта,

якого конструкція препринту не використовує, і топологія стає залежною від коефіцієнтів. При $r = 64$ такий варіант пропустив 2024 з 12 066 точкових множників для одного входу і 2002 для іншого. Наша попередня реалізація робила саме так, і це породило хибні ефекти: перетин глибин адаптивного правила і правила «чанк 1» між $r = 320$ і 336 та розкид глибини між seed порядку $10^{-4}$. У строгій конструкції немає жодного з них (підрозділи 5.4 і 5.5). Варіант із пропуском нулів далі з'являється лише як явно позначене порівняння. Жодне виправлення нашої реалізації не виявило помилки в препринті.

**Статичний аудит.** Ми переглянули всі умовні оператори в модулях, що будують слово (`conv_dag.py`, `chunk_dag.py`, `replay.py`, `rounds.py`, `toeplitz_disp.py`, `mp_exact.py`, `mp_compiler.py`, `generic_word.py`).

- Розгалуження за значенням коефіцієнта рівно одне: пропуск точкового множника з коефіцієнтом, що дорівнює нулю точно (`conv_dag.py:91`, `mp_exact.py:91`). У строгому режимі воно вимкнене. У `generic_word.py` (точні поля) його немає взагалі.
- Усі інші умови перевіряють лише структуру: чи існує вузол (`None` для доповнення нулями до степеня двійки), тип посилання (`'x'` чи `'z'`), індекси, розміри блоків і результат правила чанка, яке рахує гейти структурно.
- Пороги $|c| < 10^{-12}$ (`compiler.py`, float64) і $|c| < 10^{-30}$ (`mp_compiler.mp_compile_blocks`) трапляються лише у функціях, не використаних для жодного числа цієї статті.
- Генератор випадкових DAG у `replay.py` (рядки 215–231) використовується лише в тестах.

# 4 Методологія перевірки

## 4.1 Чотири рівні доказовості

Ми розрізняємо чотири види доказів, які не слід змішувати.

**(Д1) Доведення обчисленням.** Точне обчислення в $\mathbb{Q}(\zeta_N)$ з раціональними коефіцієнтами, де рівність є рівністю комплексних чисел. Обчислення слова на всіх базисних векторах доводить матричну тотожність для цього $r$.

**(Д2) Точна, але необхідна умова.** Точне обчислення в $\mathbb{F}_p$. Оскільки $\mathbb{F}_p$ — образ гомоморфізму кілець (твердження 1), збіг у $\mathbb{F}_p$ необхідний для комплексної тотожності, але не доводить її.

**(Д3) Числова узгодженість.** Арифметика високої точності (mpmath) і float64. Збіг при двох точностях узгоджується з точною рівністю, але не є доведенням.

**(Д4) Вимірювання конкретного слова.** Топологія, ASAP-глибина, кількість гейтів, флопи, час і пам'ять.

Їх доповнюють ще два види перевірок. *Механічні перевірки*: перекомпіляція Lean-формалізації (підрозділ 4.8) і модульні тести нашого коду; поточний повний discovery містить 123 успішні тести, зокрема регресійні для точних коренів, строгого режиму, фазового округлення і валідатора таблиць. Модульні тести самі собою не доводять жодної теореми препринту. *Аналітичне зіставлення* реалізації з текстом препринту: у підрозділі 2.4 і розділі 3 зіставлено основні позначення і перелічено всі відомі відхилення; повної построкової звірки немає.

## 4.2 Точна арифметика

Для перевірок без округлення ми реалізували два поля зі спільним інтерфейсом (`allength/exact_field.py`) і конструктор слова над довільним полем (`allength/generic_word.py`: той самий алгоритм, що й mp-версія, але без жодного пропуску множників).

**Циклотомічні поля.** Елементи $\mathbb{Q}(\zeta_N)$ — многочлени з раціональними коефіцієнтами за модулем $N$-го циклотомічного многочлена $\Phi_N$; обернення виконується розширеним алгоритмом Евкліда з перевіркою $a \cdot a^{-1} = 1$. Беремо

$$N = \operatorname{lcm}\bigl(r,\, 2^{\lceil \log_2 8r \rceil}\bigr), \tag{16}$$

щоб $u = \zeta_r$ і всі корені порядку степеня двійки $< 8r$, потрібні згортковим DAG, лежали в полі. Рівність тут — рівність комплексних чисел при вкладенні $\zeta_N \mapsto e^{2\pi i/N}$. Перевірка всіх базисних векторів є доведенням обчисленням того, що матриця точного конструктора `generic_word.py` дорівнює $F_r(u)$ для цього $r$.

**Скінченні поля.** Беремо $p$ серед найменших простих вигляду $kN + 1$, більших за $2^{61}$, і $\omega \in \mathbb{F}_p$ порядку $N$. Редукція $\zeta_N \mapsto \omega$ — гомоморфізм кільця $\mathbb{Z}[\zeta_N]$ (а не всього $\mathbb{Q}(\zeta_N)$); він продовжується на ті раціональні елементи, знаменники яких мають ненульовий образ за модулем $p$. Код обертає і перевіряє $a \cdot a^{-1} = 1$, тож ділення на образ нуля виявляється. Для коефіцієнтів нашого слова ці знаменники включають $H_k$, сталі $C$-розкладу і параметри $t_1, t_2$ розщеплених зсувів; під час точного виконання кожне обернення перевіряється на ненульовість за модулем обраного $p$. Комплексне міркування для $t_1, t_2 \neq 0$ (лема 2) не переноситься на $\mathbb{F}_p$, бо використовує додатність: образ $\mu\bar{\mu}$ там не є невід'ємним дійсним числом.

**Твердження 1.** *Нехай $\mathfrak{p}$ — ядро гомоморфізму $\mathbb{Z}[\zeta_N] \to \mathbb{F}_p$, $\zeta_N \mapsto \omega$. Якщо нерозгалужену програму над $\mathbb{Q}(\zeta_N)$ із входами в $\mathbb{Z}[\zeta_N]$ і раціональними сталими, знаменники яких взаємно прості з $p$, виконано в $\mathbb{F}_p$ і кожне обернення там застосовується до ненульового елемента, то результат у $\mathbb{F}_p$ є образом результату в $\mathbb{Q}(\zeta_N)$. Зокрема, кожна тотожність, що виконується над $\mathbb{Q}(\zeta_N)$, виконується і в $\mathbb{F}_p$.*

Доведення (локалізація в $\mathfrak{p}$) наведено в додатку Б.3. Отже, збіг у $\mathbb{F}_p$ — точна, але лише необхідна умова; кількість різних простих наведено в таблиці 2 і в описі окремого потокового тесту (підрозділ 5.1).

**Негативний контроль.** Якщо змінити один коефіцієнт у середині слова при $r = 16$, точна перевірка в $\mathbb{F}_p$ знаходить 60 розбіжностей серед 256 елементів матриці.

## 4.3 Арифметика високої точності

Ми використовуємо mpmath з 60–700 знаками. Кожну ширину повного DFT перевірено з двома точностями, що відрізняються на 100 знаків (таблиця 3). Якщо реалізований оператор дорівнює $F_r(u)$, похибка має зменшитися в $10^{100}$ разів. Відносна похибка

$$\eta = \frac{\|y - F_r x\|_2}{\|F_r x\|_2}, \tag{17}$$

де $F_r x$ обчислюється прямою сумою зі степенями $u$. Для DFT нормовані пряма і зворотна похибки збігаються, бо $\|F_r x\|_2 = \sqrt{r}\,\|x\|_2 = \|F_r\|_2 \|x\|_2$; тому ми наводимо лише одну. «Втрачено знаків» — кількість робочих знаків мінус $-\log_{10}\eta$.

### 4.4 Коефіцієнти оператора і вектори даних

Ми розрізняємо дві речі.

- *Коефіцієнти оператора $h$.* Для DFT вони фіксовані: $h_k = 1/H_k$ для заданих $r$ і $u$. Випадкові $h$ перевіряють загальний Тепліцевий компілятор, але не відтворюють структури випадку Фур'є; тести одного не підтверджують іншого.
- *Вектори даних.* У $\mathbb{Q}(\zeta_N)$ і $\mathbb{F}_p$ базисні і вибіркові випадкові вектори перелічено в таблиці 2. У mp всі базисні вектори використано при $r = 7, 8, 9, 15, 16, 31, 32, 33, 63, 64, 128$, а три випадкові — при $r = 96, 127, 256$ (таблиця 3). Дійсна і уявна частини випадкових векторів рівномірні на $[-1, 1]$ у mp; у $\mathbb{F}_p$ елементи рівномірні по полю.

Тепліцеве слово з випадковими $h$ (таблиця 8): дійсна і уявна частини $h_k$ і $x_k$ рівномірні на $[-1, 1]$, $h_0$ замінено на $|h_0| + 1$, seed $= r$. Строгі вимірювання глибини (таблиці 10, 11 і 18): $h_0$ замінено на $|h_0| + 1$, решта $h_k = \mathcal{N}(0, 1) + i\,\mathcal{N}(0, 1)$, seed $= r$; структуру додатково перевірено ще на двох входах ($h =$ ones та інший seed).

### 4.5 Похибка по етапах, ріст і точність фаз

Ланцюжок (9) має п'ять етапів: $D_H$, $T^{\mathsf{T}}$, $D_{\text{mid}}$, $T$, $D_H$. Еталонний прогін зі 150 додатковими знаками дає точні виходи $s_k$ кожного етапу. Для кожного етапу ми вимірюємо: *локальну втрату* (етап, застосований на робочій точності до точного входу $s_{k-1}$), *накопичену втрату*, *ріст* проміжних значень ($\log_{10}$ максимального модуля координати всередині етапу, поділеного на максимальний модуль його входу) і норму виходу. Обумовленість $T$ і $D_H$ обчислюється через SVD у mpmath.

**Розклад росту.** Ми будуємо слово з тегом (ширина блоку $v$, фаза) на кожному шарі (`causal_levels.py`); фаза — лист ($v = 1$), пряме слово (блок без допустимого чанка) або міжблоковий чанк (реплей displacement-DAG, побудованого з оберненого ряду $g$). Відтворюючи слово на еталонному векторі, кожному шару приписуємо додатний приріст $\log_{10}$ поточного максимуму модуля координати. Сума цих приростів телескопічно дає $\log_{10}\bigl(\text{глобальний максимум}/\|\text{вхід}\|_\infty\bigr)$, тож розклад адитивний за *ростом* (а не безпосередньо за похибкою) по рівнях і фазах.

**Контроль точності фаз.** У `phase_precision.py` коефіцієнти і вхід готуються на еталонній точності DPS + 150 і фіксуються між прогонами. Для $T^{\mathsf{T}}$ вхід — $D_H x$; для $T$ — еталонний $D_{\text{mid}} T^{\mathsf{T}} D_H x$, тобто фактичний вхід цього етапу в ланцюжку DFT. Операції вибраної фази виконуються на еталонній точності, решта — на DPS. У зсуві добуток коефіцієнта на координату і наступне додавання округлюються окремо, як у звичайному виконанні слова; це не злитий добуток із додаванням. Порівняння ведеться з виконанням усього слова на еталонній точності. Контроль вимірює вплив точності виконання фаз; похибки підготовки коефіцієнтів він не вимірює, а еталон залишається скінченним високоточним наближенням. Для $r = 128$ збільшення запасу зі 150 до 200 знаків не змінило округлених результатів. Обидві умови вище (окреме округлення; фактичний вхід етапу) легко порушити; попередня версія нашого контрольного скрипту порушувала обидві, і всі наведені тут числа отримано виправленим скриптом.

### 4.6 Глибина як аналог раундів

ASAP-розклад ставить кожен шар у найраніший раунд після всіх попередніх шарів, що торкаються його координат. Це виміряна глибина саме цього слова і верхня межа того, чого може досягти розклад цього слова. Вона є вимірюваним аналогом «змішаних раундів» з

`loc:toeplitz`, а не тотожним поняттям, і оптимальності розкладу ми не доводили. Великі ширини обчислюються потоково (`strict_tables.py`), без зберігання слова в пам'яті.

### 4.7 Одиниці вартості

Гейт — один зсув $x_{\mathrm{dst}} \mathrel{+}= c\, x_{\mathrm{src}}$ або одне масштабування $x_d \mathrel{*}= c$. У дійсних флопах при звичайному комплексному множенні масштабування коштує 6 операцій, а зсув — 8. Спеціальні коефіцієнти можуть коштувати менше; цього ми не враховуємо. Еталони в тих самих одиницях:

- радікс-2 FFT: $5r \log_2 r$;
- Тепліцеве множення через циркулянтне вкладення довжини $2r$: два FFT довжини $2r$ і $2r$ точкових добутків, тобто $2 \cdot 5 \cdot 2r \log_2(2r) + 12r$ (спектр коефіцієнтів підготовлено заздалегідь);
- пряме трикутне Тепліцеве множення: $r(r+1)/2$ комплексних множень з додаванням, тобто $4r(r+1)$.

DFT через слово коштує два Тепліцеві слова плюс три діагональні множники ($18r$ флопів). Форма з $C$-слотами коштує ще більше: кожен зсув перетворюється на 8 діагональних множників і 6 $C$-слотів. Ці числа не є часом виконання алгоритму препринту і не оцінюють його. У препринті локальне слово ніколи не виконується окремо на одному векторі: його слоти тензорно поєднуються по всіх осях і виконуються мережею, тож межа рахує слоти, а не гейти (`synchronization.tex:74--82`); крім того, час у препринті включає підготовку скалярів і роботу з індексами.

### 4.8 Перекомпіляція Lean і аудит формулювання

**Збірка.** Ми зібрали мінімальний проєкт: Lean 4.34.1, Mathlib на фіксованому коміті `d13f23b723b8a846827a245b89c10fc7d3f11612` і копії папок `FourierTransform` і `FourierCircuit` з репозиторію OpenAI. Замикання імпортів `OAI.Computability.FourierTransform.Main` має 89 модулів: 88 з `FourierTransform` і `FourierCircuit/Core.lean`; зовнішня залежність — Mathlib. Первісна повна збірка виконала 9012 задач без помилок і попереджень про `sorry`; це число включає й модулі, яких `Main` не імпортує. Усі 89 модулів його замикання окремо перевірено повторно. `#print axioms` для `transform_main`, `convolution_main` і обох варіантів `_order` показує лише `propext`, `Classical.choice` і `Quot.sound`.

**Що доведено.** `transform_main` має тип `DFTGoal` (`Goal.lean`): існують фіксовані програми `order` і `solve` та функція $W$, незалежна від вектора даних, для яких виконуються `DFTProgram` і `TimeBounds W`. Для кожного $n > 0$ `order` за довжиною $n$ видає порядок кореня $D$ з $0 < D < 1024\, n^3$; `solve` на вході $(n, \zeta_D, x)$ повертає рівно `dft n x` для кожного $x \in \mathbb{C}^n$. Умова `d.work` $+ 1 +$ `result.work` $\le W(n)$ включає підготовку порядку, одну операцію подання кореня і саме перетворення. Поліноміальна межа $(n+2)^{\texttt{cBound}}$ обмежує цілочисельний пік обох програм, $D$ і $W(n) + 10(n+2)$; це не окрема явна межа розмірів масивів. `TimeBounds W` вимагає $W = O(\texttt{paperTime})$, $W = O(\texttt{decimalTime})$ і $W = o(n \log n)$, де $\texttt{paperTime}(n) = n(\ln n)^{\theta}(\ln \ln n)^{4-\theta}$ з тими самими $m$, $W_*$, $\Delta$ і $\theta$, що й у теоремі 1. `transform_main_order` додатково дає $D = \texttt{receipt}(n) < 8\, n^3$; формула `receipt` відрізняється від $D_*$ у `eq:master-root` препринту, але задовольняє потрібну межу.

**Модель вартості (`RAM.lean`).** Кожен атом коштує одну одиницю роботи: комплексні add/sub у межах виду регістра; `scale` (скаляр×скаляр під час підготовки або скаляр×дані); `inv` лише на скалярі, з умовою ненульовості у `valid`; сталі 0 і 1; цілочисельні літерали і add/sub/mul/div/mod/lt; `len`/`look` та `id`/`fst`/`snd`. Комбінатори, ітерації і рівні рекурсії оплачуються додатково; побудова масиву довжини $\ell$ коштує $1 + \ell$. Довільної комплексної сталої і атома перетворення між $\mathbb{C}$ і $\mathbb{N}$ немає; розгалуження використовують лише цілі числа. Добуток двох значень даних (`cross`) дозволено лише для згортки (`allow = true`); у DFT його немає (`Prog false`). Обернення має окремий запис із `work` $= 1$ і `valid` $x \neq 0$, а не виклик `Bill.one`. Типи розділяють скаляри і дані.

**Построкова звірка означень.** Ми построково звірили означення (не тіла доведень): кожну ознаку моделі з `sec:model` препринту і кожну частину `thm:main` зіставлено з конкретною конструкцією Lean (`release/lean/MODEL_AUDIT.md`; підсумок у додатку B). Збіг повний за всіма переліченими ознаками: окремі регістри скалярів і даних (`Paint`); одинична вартість кожної операції поля; множення входу лише на підготовлені скаляри; ділення лише на скалярах з ненульовим знаменником, зафіксованим у `valid`, а не перевіреним тестом; відсутність інструкції, що створює довільну комплексну сталу (корінь — вхід); одиничні цілочисельні операції з поліноміально обмеженим піком; єдиний поданий корінь порядку $0 < D < 1024\,n^3$; точна комплексна арифметика без поля похибки; відсутність добутку даних для DFT; і межа часу `paperTime` з натуральними логарифмами. Підтверджено також ланцюг означень кореня і матриці: `zeta` $n = \exp(2\pi i/n)$, `fourierMatrix` $n\,j\,k =$ `zeta` $n^{jk}$, а `DFTProgram` вимагає `result.val` $=$ `dft` $n\,x = F_n x$. Послаблення головної цілі не знайдено.

**Межі звірки.** Адекватність обліку `Bill` щодо фізичної word-RAM залишається модельним судженням; окремої теореми симуляції немає. Тіл доведень ми построково не перечитували; їхню коректність для цих означень перевіряє ядро Lean. Відтворення компіляції чужої формалізації не є незалежним доведенням головної теореми.

**Як читати збіг.** Формалізація використовує іншу, меншу мережу: 32 двійкові координати на множник, 4060 трійок і межу $2^{50}$. Внутрішні леми `lunch_big` і `alpha_theta` доводять $W = O\big(n(\ln n)^{\alpha}(\ln\ln n)^2\big)$ з $\alpha = 1 - 2 \cdot 10^{-11} < \theta$, звідки випливає потрібна верхня межа `paperTime`. Цю сильнішу внутрішню межу окремо перевірено коротким Lean-твердженням, зібраним із цих лем. Отже, *формулювання* головного результату доведено, але конкретна мережа з `network.tex`, її $\Delta$ і `net:tensor-bound` цією іншою конструкцією не перевіряються. Синтез і загальна алгебра з додатка препринту до формалізації не входять; за вступом препринту цей шлях не потрібен для заявленого показника.

### 4.9 Програмне забезпечення і середовище

Python 3.12.3, numpy 1.26.4, mpmath 1.3.0, matplotlib 3.11.1, scipy 1.11.4 (обов'язкова залежність: `allength/compiler.py` використовує `scipy.optimize.least_squares`). Intel Core i9-14900HX, 32 потоки, 31 ГіБ RAM, Linux 7.0. Переносний набір залежностей — `release/requirements-lock.txt` (встановлюється в чисте віртуальне середовище); повний freeze середовища розробки — `release/requirements-full-freeze.txt`; див. також `release/ENVIRONMENT.txt`.

### 4.10 Конвеєр відтворення

Усі числа в розділі 5 отримано в повному прогоні `full-20261013-r3` конвеєра `release/run_all.sh full`, який виконав усі 135 кроків. Кожен прогін копіює джерела, отримує сирі результати, агрегує їх у CSV-файли лише цього прогону, звіряє кожну таблицю рукопису з цими CSV-файлами і заново будує рисунки з тих самих CSV-файлів; він

**Таблиця 2:** Точні перевірки: кожен елемент результату порівнюється з $u^{ij}$ на рівність. Рядки $C$-слотів використовують розщеплення (8) і спряження через DAG з оберненими коренями.

| $r$ | Поле | Вектори | Перевірено елементів | Розбіжностей |
|---|---|---|---|---|
| 4 | $\mathbb{Q}(\zeta_{32})$, степінь 16 | 4 базисних | 16 | 0 |
| 7 | $\mathbb{Q}(\zeta_{448})$, степінь 192 | 7 базисних | 49 | 0 |
| 8 | $\mathbb{Q}(\zeta_{64})$, степінь 32 | 8 базисних | 64 | 0 |
| 9 | $\mathbb{Q}(\zeta_{1152})$, степінь 384 | 9 базисних | 81 | 0 |
| 12 | $\mathbb{Q}(\zeta_{384})$, степінь 128 | 12 базисних | 144 | 0 |
| 15 | $\mathbb{Q}(\zeta_{1920})$, степінь 512 | 15 базисних | 225 | 0 |
| 16 | $\mathbb{Q}(\zeta_{128})$, степінь 64 | 16 базисних | 256 | 0 |
| 32 | $\mathbb{Q}(\zeta_{256})$, степінь 128 | 32 базисних | 1024 | 0 |
| 9, 15, 16, 31, 32, 33, 63, 64, 127, 128 | $\mathbb{F}_p$, по три прості $p$ | усі базисні | від 81 до 16 384 на кожне $p$ | 0 |
| 255, 256 | $\mathbb{F}_p$, по два прості $p$ | 3 випадкових | 765 і 768 на кожне $p$ | 0 |
| 512 | $\mathbb{F}_p$, два прості $p$ | 2 випадкових | 1024 на кожне $p$ | 0 |
| 4, 8, 16 (C-слоти + спряження через DAG) | $\mathbb{Q}(\zeta_{32})$, $\mathbb{Q}(\zeta_{64})$, $\mathbb{Q}(\zeta_{128})$ | усі базисні | 16, 64, 256 | 0 |
| 16, 31, 32, 33, 64 (C-слоти + спряження через DAG) | $\mathbb{F}_p$ | усі базисні | 256, 961, 1024, 1089, 4096 | 0 |
| 128 (C-слоти + спряження через DAG) | $\mathbb{F}_p$ | 2 випадкових | 256 | 0 |

записує маніфест команд, кодів завершення і хешів. Цілісність джерел, журналів, виходів, середовища й агрегації сирих даних у CSV цього прогону перевірено без підстановки архівних даних. Два винятки зазначено там, де вони трапляються: попереднє вимірювання часу (таблиця 17) і окремий потоковий $\mathbb{F}_p$-тест при $r = 1024$, який не входить до 135 кроків. Доступність описано в розділі 8.

# 5 Результати

## 5.1 Правильність повного ланцюжка

Таблиця 2 доводить обчисленням, що для $r = 4, 7, 8, 9, 12, 15, 16$ і 32 матриця точного конструктора `generic_word.py` дорівнює $F_r(u)$; для $r = 4, 8$ і 16 це виконується й для шляху через $C$-слоти з розщепленням і спряженням через DAG. Для більших $r$ точні перевірки в $\mathbb{F}_p$ є необхідною умовою і узгоджуються з рівністю. Перевірені ширини в $\mathbb{F}_p$: $r = 9, 15, 16, 31, 32, 33, 63, 64, 127, 128$ (не всі цілі до 128; включно з непарними і простими ширинами).

**Потоковий $\mathbb{F}_p$-тест при $r = 1024$.** Матеріалізований $\mathbb{F}_p$-тест при $r = 1024$ не завершився через нестачу пам'яті. Окремий потоковий тест 2026-10-14 (`stream_exact_dft.py`) завершився: $N = 8192$, $p = 2305843009213800449$ і $p = 2305843009213931521$; ті самі два цілочисельні випадкові вектори в кожному полі, 2048 координат на поле, разом 4096, 0 розбіжностей. Генератор відтворює рекурсію строгого слова, застосовує кожен шар і від-

**Таблиця 3:** Повний ланцюжок у mp з двома точностями, що відрізняються на 100 знаків.

| $r$ | Вектори | Точність, знаків | Макс. відн. похибка | Втрачено знаків | $\log_{10}\max_k\lvert H_k\rvert$ | $\log_{10}\max_k\lvert 1/H_k\rvert$ |
|---|---|---|---|---|---|---|
| 7 | 7 базисних | 60 / 160 | $1{,}5\cdot10^{-60}$ / $1{,}9\cdot10^{-160}$ | 0,2 / 0,3 | 0,9 | 0,1 |
| 8 | 8 базисних | 60 / 160 | $2{,}5\cdot10^{-60}$ / $2{,}8\cdot10^{-160}$ | 0,4 / 0,5 | 1,0 | 0,1 |
| 9 | 9 базисних | 60 / 160 | $4{,}2\cdot10^{-60}$ / $2{,}8\cdot10^{-160}$ | 0,6 / 0,4 | 1,1 | 0,2 |
| 15 | 15 базисних | 60 / 160 | $5{,}2\cdot10^{-58}$ / $6{,}3\cdot10^{-158}$ | 2,7 / 2,8 | 1,6 | 0,5 |
| 16 | 16 базисних | 60 / 160 | $1{,}4\cdot10^{-57}$ / $2{,}2\cdot10^{-157}$ | 3,1 / 3,3 | 1,7 | 0,5 |
| 31 | 31 базисних | 60 / 160 | $1{,}0\cdot10^{-49}$ / $1{,}5\cdot10^{-149}$ | 11,0 / 11,2 | 2,9 | 1,4 |
| 32 | 32 базисних | 60 / 160 | $1{,}9\cdot10^{-49}$ / $2{,}1\cdot10^{-149}$ | 11,3 / 11,3 | 3,0 | 1,5 |
| 33 | 33 базисних | 60 / 160 | $1{,}5\cdot10^{-48}$ / $1{,}9\cdot10^{-148}$ | 12,2 / 12,3 | 3,1 | 1,6 |
| 63 | 63 базисних | 100 / 200 | $1{,}8\cdot10^{-71}$ / $2{,}4\cdot10^{-171}$ | 29,3 / 29,4 | 5,3 | 3,5 |
| 64 | 64 базисних | 120 / 220 | $9{,}1\cdot10^{-91}$ / $9{,}1\cdot10^{-191}$ | 30,0 / 30,0 | 5,4 | 3,6 |
| 96 | 3 випадкових | 200 / 300 | $3{,}9\cdot10^{-153}$ / $6{,}5\cdot10^{-253}$ | 47,6 / 47,8 | 7,7 | 5,7 |
| 127 | 3 випадкових | 250 / 350 | $9{,}8\cdot10^{-185}$ / $1{,}8\cdot10^{-284}$ | 66,0 / 66,3 | 10,0 | 7,9 |
| 128 | 128 базисних | 250 / 350 | $1{,}1\cdot10^{-183}$ / $1{,}5\cdot10^{-283}$ | 67,0 / 67,2 | 10,0 | 7,9 |
| 256 | 3 випадкових | 450 / 550 | $9{,}6\cdot10^{-309}$ / $7{,}7\cdot10^{-409}$ | 142,0 / 141,9 | 19,2 | 16,8 |

**Таблиця 4:** Шлях через $C$-слоти з розщепленням і спряженням через DAG у mp (`mp_check_cslots.py --conj-dag`).

| $r$ | Вектори | Точність, знаків | Блоків зсуву на вектор | Макс. відносна похибка | Втрачено знаків |
|---|---|---|---|---|---|
| 7 | 7 базисних | 60 | 84 | $7{,}1\cdot10^{-60}$ | 0,9 |
| 9 | 9 базисних | 60 | 144 | $2{,}2\cdot10^{-59}$ | 1,3 |
| 15 | 15 базисних | 60 | 8 260 | $4{,}9\cdot10^{-57}$ | 3,7 |
| 31 | 31 базисних | 60 | 52 260 | $8{,}2\cdot10^{-47}$ | 13,9 |
| 33 | 33 базисних | 60 | 59 232 | $1{,}6\cdot10^{-45}$ | 15,2 |
| 63 | 63 базисних | 100 | 250 852 | $1{,}3\cdot10^{-65}$ | 35,1 |
| 64 | 64 базисних | 120 / 220 | 258 944 | $5{,}5\cdot10^{-85}$ / $7{,}1\cdot10^{-185}$ | 35,7 / 35,9 |
| 128 | 3 випадкових | 250 / 350 | 3 712 768 | $4{,}3\cdot10^{-171}$ / $5{,}0\cdot10^{-271}$ | 79,6 / 79,7 |
| 256 | 1 випадковий | 450 / 550 | 20 205 056 | $4{,}8\cdot10^{-281}$ / $2{,}4\cdot10^{-381}$ | 169,7 / 169,4 |

кидає його; транспонований прохід обертає порядок шарів і переставляє кінці зсувів. На малих ширинах потоки звірено з матеріалізованим словом і виконано негативні контролі (неправильний порядок, переставлені кінці, зіпсований коефіцієнт). При $r = 1024$ кожне слово має 4 796 928 шарів і 64 639 488 гейтів. Повторний прогін тривав 431,9 с з піковим RSS (GNU `time`) 37,91 МіБ; його хеші виходів і потоків збігаються з першим прогоном (450,95 с, 37,48 МіБ). Хеші джерел записано на початку і наприкінці. Цей додатковий експеримент не входить до 135 кроків; він перевіряє необхідну умову на двох векторах, а не всю матрицю, комплексну рівність чи $C$-слоти при $r = 1024$.

Для всіх ширин похибка зменшується приблизно на 100 десяткових порядків при підвищенні точності на 100 знаків; для повного DFT у таблиці 3 десятковий виграш лежить у межах 99,7–100,2 (наприклад, 99,7 при $r = 127$ і 100,1 при $r = 256$). Це узгоджується з точною рівністю і не виявляє систематичної помилки реалізації, але не є доведенням рівності для цих $r$; його дають лише точні перевірки таблиці 2.

У таблиці 4 кожен блок зсуву складається з 4 діагональних множників і 3 $C$-слотів, а кожен зсув слова дає два блоки. Для спряжених коефіцієнтів ми вимірюємо $\max_\mu|\bar{\mu}_{\mathrm{DAG}} - \mathrm{conj}(\mu)|/\max(1,|\mu|)$: це нормоване відхилення становить $3{,}3\cdot10^{-89}$ при $r = 63$ і 100 знаках та

**Таблиця 5:** Повний ланцюжок у float64.

| $r$ | Вектори | Макс. відн. похибка | Медіана |
|---|---|---|---|
| 8 | 8 базисних | $9{,}1 \cdot 10^{-15}$ | $2{,}8 \cdot 10^{-15}$ |
| 16 | 16 базисних | $3{,}3 \cdot 10^{-12}$ | $6{,}0 \cdot 10^{-13}$ |
| 24 | 24 базисних | $3{,}4 \cdot 10^{-8}$ | $8{,}0 \cdot 10^{-9}$ |
| 32 | 32 базисних | $3{,}4 \cdot 10^{-4}$ | $3{,}4 \cdot 10^{-5}$ |
| 48 | 6 випадкових | $3{,}2 \cdot 10^{7}$ | $8{,}7 \cdot 10^{6}$ |
| 64 | 6 випадкових | $8{,}1 \cdot 10^{23}$ | $6{,}7 \cdot 10^{23}$ |
| 96 | 6 випадкових | $6{,}0 \cdot 10^{56}$ | $3{,}7 \cdot 10^{56}$ |

**Таблиця 6:** Обумовленість трикутного і діагонального множників (скінченні значення) і головний член $cr$.

| $r$ | $\log_{10}\kappa_2(T)$ | $\log_{10}\kappa_2(D_H)$ | Головний член $cr$ |
|---|---|---|---|
| 16 | 2,22 | 2,25 | 2,24 |
| 32 | 4,48 | 4,49 | 4,49 |
| 48 | 6,72 | 6,73 | 6,73 |
| 96 | 13,46 | 13,47 | 13,47 |
| 128 | 17,95 | 17,96 | 17,96 |
| 256 | — | 35,92 | 35,92 |
| 1024 | — | 143,673 | 143,673 |

$1{,}2 \cdot 10^{-225}$ при $r = 128$ і 250 знаках, а в частині прогонів дорівнює нулю; це не абсолютна похибка коефіцієнтів великого модуля. Шлях через $C$-слоти втрачає більше знаків, ніж слово зі зсувами: на 1–3 знаки при $r \le 33$, близько 6 при $r = 64$ (35,7 проти 30,0), близько 13 при $r = 128$ (79,6 проти 67,0) і близько 28 при $r = 256$ (169,7 проти 142,0). Дві точності в таблиці 4 є лише для $r = 64, 128$ і 256; для всіх трьох похибка зменшується приблизно на 100 десяткових порядків, включно з $r = 256$ (від $4{,}8 \cdot 10^{-281}$ при 450 знаках до $2{,}4 \cdot 10^{-381}$ при 550 знаках, виграш 100,30 порядку; втрачено знаків 169,7 і 169,4). Прогін при $r = 256$ використовує один випадковий вектор; повторне вимірювання 2026-10-14 при 550 знаках зайняло 448 с (7,5 хв) на побудову двох слів і 2326 с (39 хв) загалом. Це час нашої реалізації; стороннє навантаження не контролювали.

## 5.2 Числова нестійкість

Для самого оператора $F_r^* F_r = rI$, тобто $\kappa_2(F_r) = 1$. Тому зростання похибки float64 у таблиці 5 не можна пояснити поганою обумовленістю DFT: нестійкий у скінченній точності саме обчислювальний шлях. У точній арифметиці той самий шлях дає точний результат (таблиця 2).

**Обумовленість множників.** У таблиці 6 другий стовпець — чисельно обчислена скінченна $\kappa_2(T)$, третій — скінченна $\kappa_2(D_H)$, четвертий — її головний асимптотичний член $cr$, де

$$c = \frac{\mathrm{Cl}_2(\pi/3)}{\pi \ln 10} = 0{,}14030576. \tag{18}$$

Тут $\log_{10}\kappa_2(D_H) = \log_{10}\bigl(\max_k |H_k| / \min_k |H_k|\bigr)$ — точна скінченна величина (різниця часткових сум). Оскільки $\log|H_k| = \sum_{s \le k} \log\bigl(2\sin(\pi s/r)\bigr)$, різниця між максимумом (при $k \approx 5r/6$) і мінімумом (при $k \approx r/6$) дорівнює

$$\log_{10}\kappa_2(D_H) = \frac{r}{\pi}\,\frac{\mathrm{Cl}_2(\pi/3)}{\ln 10} + o(r), \tag{19}$$

де $\mathrm{Cl}_2(\pi/3) \approx 1{,}01494$ — значення функції Клаузена (твердження 2 у додатку Б.2). Це головний асимптотичний член, а не точна формула. Скінченне значення відрізняється від $cr$ на рівні $o(r)$: при $r = 16$ воно дорівнює 2,25 проти $cr = 2{,}24$, при $r = 1024$ — 143,6732 проти 143,6731 (різниця $10^{-4}$). У випадку Фур'є обумовленість $T$ практично дорівнює обумовленості $D_H$ на перевірених ширинах до 128 (найбільше і найменше сингулярні значення $T$ взаємно обернені з точністю до $10^{-2}$); точної формули для $T$ ми не доводили. Отже, обумовленість Тепліцевого множника росте приблизно як $10^{cr}$, $c \approx 0{,}1403$; це властивість природного порядку вузлів у ньютоновій формі (підрозділ 2.6), а не нашої реалізації.

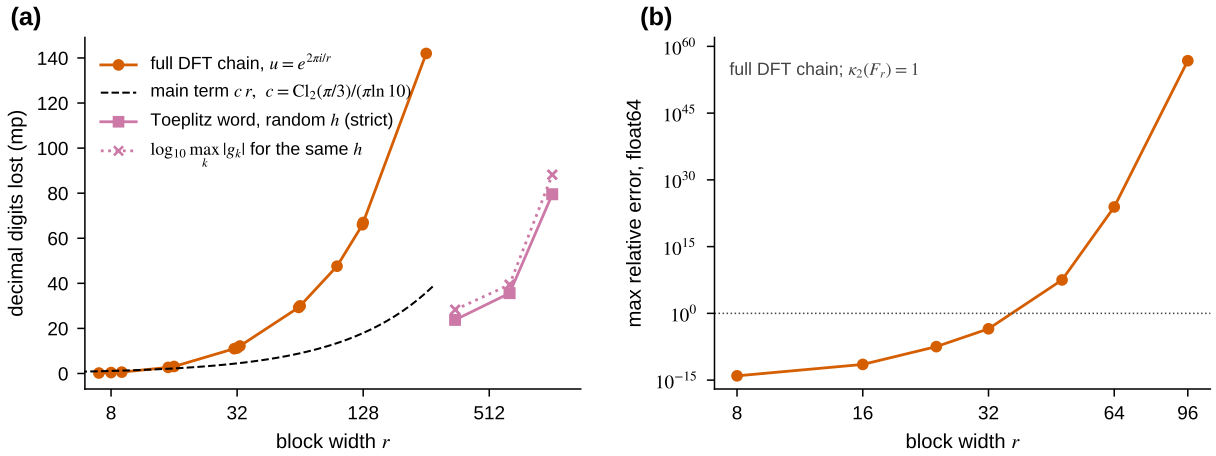



**Рис. 1:** (a) Втрачені десяткові знаки в mp: повний ланцюжок DFT (таблиця 3, усі ширини 7–256), Тепліцеве слово з випадковими $h$ (таблиця 8) разом із $\log_{10} \max_k |g_k|$ для тих самих $h$ і головний асимптотичний член $cr$, $c = \mathrm{Cl}_2(\pi/3)/(\pi \ln 10)$. (b) Відносна похибка повного ланцюжка у float64 (таблиця 5); $\kappa_2(F_r) = 1$. Підписи на рисунках англійською.

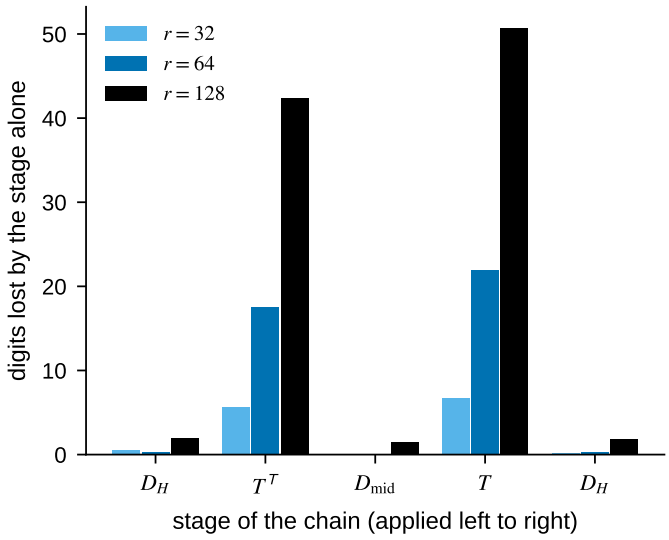



**Рис. 2:** Локальна втрата знаків по етапах ланцюжка $D_H$, $T^{\mathsf{T}}$, $D_{\mathrm{mid}}$, $T$, $D_H$ при $r = 32, 64, 128$ (таблиця 7).

**Втрата по етапах (таблиця 7 і рисунок 2).** (1) Діагональні етапи майже не втрачають знаків ($\leq 2$); уся втрата припадає на два Тепліцеві слова. (2) Для $T^{\mathsf{T}}$ локальна втрата майже дорівнює росту проміжних значень (5,6 і 6,5; 17,5 і 19,1; 42,4 і 43,9): похибка поводиться як $\varepsilon$, помножене на найбільшу проміжну величину. (3) Для $T$ локальна втрата перевищує ріст; вона порівняння з $\log_{10} \kappa_2(T)$ плюс ріст (при $r = 64$: $9{,}0 + 9{,}2 \approx 18$ проти 21,9), тобто тут додається підсилення похибки входу самим оператором. (4) Повна втрата (таблиця 3) становить 2,5, 3,3, 3,7 і 4,0 одиниці $\log_{10} \kappa_2(T)$ при $r = 32, 64, 128, 256$.

**Де виникає ріст (таблиця 14 у додатку A).** Розклад росту з підрозділу 4.5 описує ріст, який може підсилювати округлення в $T^{\mathsf{T}}$ (де локальна втрата майже дорівнює росту); для $T$ втрата перевищує ріст, тож там розклад описує лише ріст. Його суми збігаються зі стовпцем росту таблиці 7 ($T^{\mathsf{T}}$: 6,5, 19,1, 43,9; $T$: 3,0, 9,2, 21,7). (1) Увесь приріст рекордного максимуму $T^{\mathsf{T}}$ припадає на фазу міжблокових чанків, тобто на реплей displacement-DAG, побудованих з оберненого ряду $g$, а не на листові чи прямі шари. Для $T$ фаза чанків дає 67–84%, решту — пряме трикутне слово базових блоків ширини 8 (нижче $v = 10$ допустимого

**Таблиця 7:** Втрата знаків по етапах (одна випадкова $x$). Локальна: етап на робочій точності, застосований до точного входу; ріст: $\log_{10}$ відношення максимального модуля проміжної координати до максимального модуля входу етапу.

| $r$ | Етап | Локальна втрата | Накопичена втрата | Ріст | $\log_{10}$ норми виходу |
|---|---|---|---|---|---|
| 32 | $D_H$ | 0,5 | 0,5 | 3,0 | 2,5 |
| 32 | $T^{\mathsf{T}}$ | 5,6 | 5,6 | 6,5 | 4,1 |
| 32 | $D_{\text{mid}}$ | 0,0 | 7,0 | −0,4 | 3,6 |
| 32 | $T$ | 6,7 | 9,4 | 3,0 | 1,9 |
| 32 | $D_H$ | 0,2 | 10,9 | −1,4 | 0,8 |
| 64 | $D_H$ | 0,3 | 0,3 | 5,4 | 4,9 |
| 64 | $T^{\mathsf{T}}$ | 17,5 | 18,0 | 19,1 | 8,9 |
| 64 | $D_{\text{mid}}$ | −0,3 | 20,9 | −0,7 | 8,1 |
| 64 | $T$ | 21,9 | 26,5 | 9,2 | 4,0 |
| 64 | $D_H$ | 0,3 | 29,1 | −3,5 | 0,9 |
| 128 | $D_H$ | 2,0 | 2,0 | 9,8 | 9,4 |
| 128 | $T^{\mathsf{T}}$ | 42,4 | 42,3 | 43,9 | 17,7 |
| 128 | $D_{\text{mid}}$ | 1,5 | 48,6 | −1,0 | 16,8 |
| 128 | $T$ | 50,7 | 59,8 | 21,7 | 8,4 |
| 128 | $D_H$ | 1,9 | 66,3 | −7,8 | 1,1 |

**Таблиця 8:** Тепліцеве слово, випадкові $h$, строгий режим.

| $r$ | Точність, знаків | Макс. відносна похибка | Втрачено знаків | $\log_{10} \max_k \lvert g_k \rvert$ |
|---|---|---|---|---|
| 352 | 300 / 400 | $5{,}8 \cdot 10^{-277}$ / $5{,}5 \cdot 10^{-377}$ | 23,8 / 23,7 | 28,3 |
| 640 | 400 / 500 | $4{,}4 \cdot 10^{-365}$ / $5{,}6 \cdot 10^{-465}$ | 35,6 / 35,7 | 39,4 |
| 1024 | 600 / 700 | $3{,}5 \cdot 10^{-521}$ / $1{,}7 \cdot 10^{-621}$ | 79,5 / 79,2 | 88,3 |

чанка немає, підрозділ 3.4). (2) Ріст розподілений по кількох рівнях рекурсії, а не зосереджений на одному. У $T^{\mathsf{T}}$ найбільший внесок дає зовнішній рівень $v = r$ (49–66%); у $T$ внесок зміщується до середніх рівнів. Нульовий приріст на якомусь рівні (наприклад, $v = 16$ для $T^{\mathsf{T}}$ при $r = 128$) означає лише, що там не виникає нового рекорду, а не що рівень не впливає на похибку. (3) Отже, великі проміжні величини локалізовано в displacement-реплеї чанків (ріст оберненого ряду $g$, підрозділ 5.3), і вони накопичуються по рівнях рекурсії.

**Контроль точності фаз (таблиця 15 у додатку A).** Підвищення точності лише фази чанків зменшує втрату $T^{\mathsf{T}}$ приблизно на 14,5 і 37,5 знака при $r = 64$ і 128; підвищення точності лише прямої фази майже не впливає. Для $T$ зменшення становить приблизно 10,3 і 27,9 знака, але залишається втрата 11,6 і 22,6 знака. Отже, для перевірених входів і ширин точність чанків сильно впливає на похибку обох етапів, особливо $T^{\mathsf{T}}$. Це не адитивний розклад похибки: округлення у фазах і подальше підсилення взаємодіють, і залишок не можна назвати лише перенесенням без окремого контролю. Експеримент охоплює один випадковий вектор (seed $= r$) на ширину, фіксовані коефіцієнти DFT і $r = 64, 128$; інших розподілів коефіцієнтів і ширших випадків він не охоплює.

### 5.3 Тепліцеве слово з випадковими коефіцієнтами

Це тест загального Тепліцевого оператора $Tx$, а не DFT, виконаний у строгому режимі з трьома векторами і двома точностями (таблиця 8). Для порівняння, варіант із пропуском нулів з одним вектором втрачав 23,6, 35,0 і 78,6 знака при $r = 352, 640, 1024$.

**Таблиця 9:** Корені $h(z)$ і ріст $g$ (`roots_h.py`; корені — власні значення супровідної матриці, відносний залишок $|h(z)|$ у найменших коренях $\le 4 \cdot 10^{-14}$). Нахил підібрано на другій половині $k$.

| $r$ | $\rho = \min\lvert\text{корінь}\rvert$ | Коренів усередині одиничного круга | Нахил $\log_{10}\lvert g_k\rvert$ | $\log_{10}(1/\rho)$ | $(r-1)\log_{10}(1/\rho)$ | Спостережений $\log_{10}\max_{k<r}\lvert g_k\rvert$ |
|---|---|---|---|---|---|---|
| 352 | 0,8256 | 179 з 351 | 0,0832 | 0,0832 | 29,2 | 28,3 |
| 640 | 0,8647 | 314 з 639 | 0,0632 | 0,0632 | 40,4 | 39,4 |
| 1024 | 0,8185 | 508 з 1023 | 0,0870 | 0,0870 | 89,0 | 88,2 |

**Механізм росту коефіцієнтів.** Ряд $g$ — обернений ряд до $h$ (`loc:reciprocal`), з якого будуються чанки; його радіус збіжності дорівнює найменшому модулю $\rho$ кореня $h(z) = \sum_k h_k z^k$. Отже, $\log_{10}|g_k|$ росте з нахилом $\log_{10}(1/\rho)$.

Нахил збігається з $\log_{10}(1/\rho)$ до четвертого знака після коми, а прогноз $(r-1)\log_{10}(1/\rho)$ відрізняється від спостереженого максимуму менш ніж на один знак (таблиця 9). Приблизно половина коренів лежить усередині одиничного круга; найменший модуль становить 0,82–0,86. Отже, для цих трьох вибірок ріст $g$ пояснюється найменшим коренем $h(z)$, а втрата знаків майже дорівнює $\log_{10}\max_k|g_k|$. Розподілу $\rho$ для інших розподілів коефіцієнтів ми не досліджували.

## 5.4 Фіксована топологія

Для кожного прогону ми обчислюємо SHA-256 від послідовності шарів, де кожен шар записано як його ASAP-раунд і список гейтів (тип M — масштабування або S — зсув, джерело, приймач). Аудит охоплює 12 ширин, 2 правила (адаптивне і чанк розміру 1) і 3 входи (seed $= r$, інший seed, $h =$ ones); повні дані наведено в таблиці 18 (додаток A). Для кожної ширини і кожного правила хеш однаковий для всіх трьох входів. Отже, у строгому режимі типи гейтів, пари координат, порядок шарів, розклад за раундами, глибина і кількість гейтів є функціями лише ширини. Разом зі статичним аудитом підрозділу 3.6 це підтверджує для нашої реалізації вимогу препринту про фіксовану топологію. Для порівняння, у варіанті з пропуском нулів глибина при $r = 64$ становила 24442 і 24452 для двох seed і 13969 для $h =$ ones.

## 5.5 Глибина

Строгу ASAP-глибину виміряно на сітці з 12 ширин: 64, 128, 256, 288, 320, 336, 352, 384, 512, 640, 768, 1024 (`strict_tables.py`; дані в `paper/data/strict_tables.csv`).

У строгому режимі адаптивне правило має глибину, не більшу, ніж чанк розміру 1, на всіх 12 ширинах сітки, і строго меншу, починаючи з $r = 128$ (таблиця 10 і рисунок 3). Перетин між 320 і 336, помітний у варіанті з пропуском нулів, зникає: там чанк розміру 1 отримував перевагу, бо в DAG для чанків розміру 1 багато точкових множників дорівнюють нулю точно ($g(1) = 6$ замість 8, $L(1) = 18$ замість 28). Стрибок відношення між 320 і 336 збігається зі стрибком $b$ від 2 до 3 і зберігається в обох режимах. Ці висновки стосуються саме цієї сітки; для проміжних $r$ ми їх не стверджуємо. Глибина не монотонна за $r$: при $r = 352$ вона менша, ніж при $r = 320$, бо більший блок допускає більший чанк.

**Модель глибини.** Ми використовуємо рекурентність

$$d(v) = d(\lfloor v/2 \rfloor) + k^2 L(b), \qquad k = \left\lceil \lfloor v/2 \rfloor / b \right\rceil, \tag{20}$$

з прямою вартістю $v(v+1)/2$ для блоків без допустимого чанка; $b$ визначається строгою таблицею $g(b)$, а $L(b)$ — кількість шарів реплею однієї пари $b \times b$ у строгому режимі

**Таблиця 10:** Строга ASAP-глибина: адаптивне правило проти чанка розміру 1. Останній стовпець — відношення глибин для залежного від значень варіанта з пропуском нулів (архівні дані).

| $r$ | $b$ | Адаптивне | Чанк 1 | Адаптивне / чанк 1 | Варіант з пропуском нулів, адаптивне / чанк 1 |
|---|---|---|---|---|---|
| 64 | 1 | 37668 | 37668 | 1,00 | — |
| 128 | 2 | 133924 | 152356 | 0,88 | — |
| 256 | 2 | 518948 | 611108 | 0,85 | 1,29 |
| 288 | 2 | 656793 | 773433 | 0,85 | 1,29 |
| 320 | 2 | 811515 | 955515 | 0,85 | 1,29 |
| 336 | 3 | 626622 | 1053510 | 0,59 | 0,89 |
| 352 | 4 | 497926 | 1156166 | 0,43 | 0,64 |
| 384 | 4 | 592581 | 1375941 | 0,43 | 0,64 |
| 512 | 4 | 1035044 | 2446116 | 0,42 | 0,65 |
| 640 | 4 | 1617915 | 3822715 | 0,42 | 0,65 |
| 768 | 4 | 1753797 | 5504709 | 0,32 | 0,49 |
| 1024 | 8 | 1682212 | 9786148 | 0,17 | 0,27 |

**Таблиця 11:** Модель строгої глибини проти виміряної ASAP-глибини (адаптивне правило).

| $r$ | Модель | ASAP | ASAP / модель |
|---|---|---|---|
| 64 | 37668 | 37668 | 1 (точно) |
| 128 | 133924 | 133924 | 1 (точно) |
| 256 | 518948 | 518948 | 1 (точно) |
| 288 | 656793 | 656793 | 1 (точно) |
| 320 | 811515 | 811515 | 1 (точно) |
| 336 | 626207 | 626622 | 1,00066 |
| 352 | 497783 | 497926 | 1,00029 |
| 384 | 592581 | 592581 | 1 (точно) |
| 512 | 1035044 | 1035044 | 1 (точно) |
| 640 | 1617915 | 1617915 | 1 (точно) |
| 768 | 1753797 | 1753797 | 1 (точно) |
| 1024 | 1682212 | 1682212 | 1 (точно) |

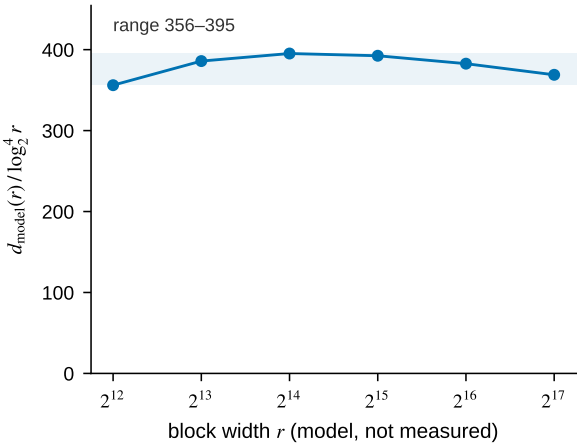



**Рис. 4:** Відношення $d_{\mathrm{model}}(r)/\log_2^4 r$ моделі строгої глибини на степенях двійки $2^{12}$–$2^{17}$; смуга позначає діапазон значень. Значення моделі, а не вимірювання. Дані: таблиця 16.

(`depth_exact_rule.py --strict`). Модель використовує виміряні $g(b)$ і $L(b)$ і йде лише гілкою $\lfloor v/2 \rfloor$; для степенів двійки обидві гілки збігаються.

Модель точно збігається з вимірюванням на 10 з 12 ширин (таблиця 11). При $r = 336$ і 352 рекурсія доходить до непарних блоків $v \geq 10$ (21 і 11), і модель, що йде лише гілкою $\lfloor v/2 \rfloor$, занижує глибину. Це прямо підтверджує, що повна рекурентність має брати максимум по обох гілках, як у доведенні препринту (6).

На степенях двійки $2^{12}$–$2^{17}$ модель дає $d_{\mathrm{model}}/\log_2^4 r$ у межах 356–395 (таблиця 16 у додатку A, рисунок 4). Ці значення отримано з моделі; ASAP-слів таких ширин ми не будували. Оскільки модель побудовано на емпіричних $g(b)$ і $L(b)$, її збіг з вимірюваннями є перевіркою узгодженості, а не незалежним доведенням $O(\log^4 r)$. Евристика порядку величини

$$g(b) \approx C\, b \log b \;\Rightarrow\; b = \Theta(v/\log v), \quad k = \Theta(\log v), \quad L(b) = \Theta(\log v), \tag{21}$$

за якою один рівень дає внесок $\Theta(\log^3 v)$, а рівнів $\log_2 r$, пояснює, чому модель поводиться як $\log^4 r$, і відповідає доведенню верхньої межі в препринті. Шість майже сталих точок узгоджуються з $O(\log^4 r)$, але не встановлюють асимптотичної рівності.

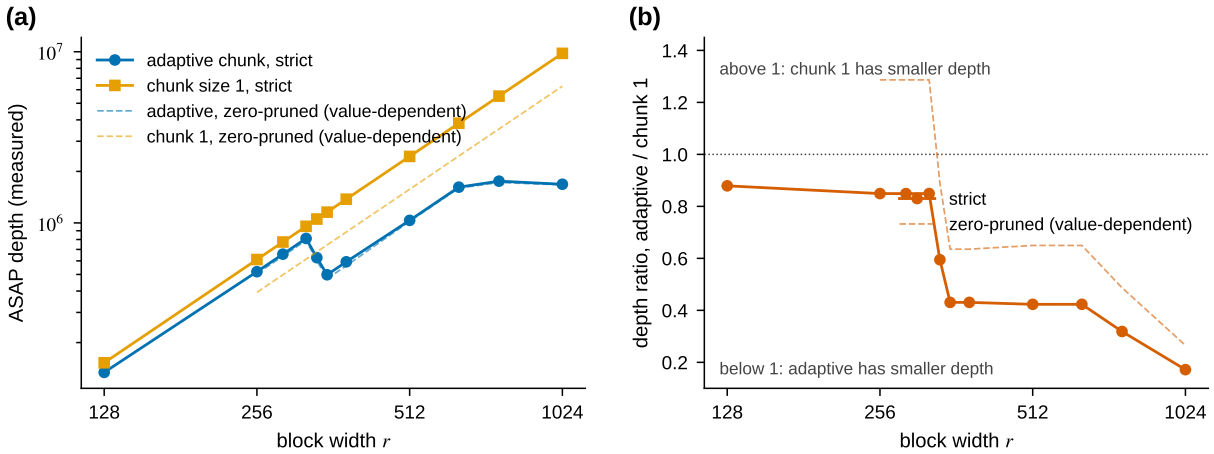



**Рис. 3:** (a) Виміряна ASAP-глибина строгого слова для адаптивного правила і чанка розміру 1 (суцільні лінії) і та сама величина для залежного від значень варіанта з пропуском нулів (тонкі штрихові лінії, архівні дані). Логарифмічні осі. (b) Відношення глибин адаптивне / чанк 1; пунктир — рівність. Нижче 1 адаптивне правило має меншу глибину. У строгому режимі відношення менше 1 на всіх ширинах, починаючи з 128; перетин варіанта з пропуском нулів біля $r \approx 330$ — артефакт пропуску нулів. Дані: таблиця 10.

**Таблиця 12:** Дійсні флопи. Відношення: DFT через слово до радікс-2 FFT, а також одне Тепліцеве слово до циркулянтного FFT і до прямого Тепліцевого множення.

| $r$ | Одне Тепліцеве слово | DFT через слово | Радікс-2 FFT | Тепліцеве через цирку-лянтний FFT | Пряме Тепліцеве | DFT через слово / FFT | Слово / цирку-лянтний FFT | Слово / пряме |
|---|---|---|---|---|---|---|---|---|
| 64 | 518 272 | 1 037 696 | 1 920 | 9 728 | 16 640 | 540 | 53 | 31 |
| 128 | 7 426 304 | 14 854 912 | 4 480 | 22 016 | 66 048 | 3 316 | 337 | 112 |
| 256 | 40 411 648 | 80 827 904 | 10 240 | 49 152 | 263 168 | 7 893 | 822 | 154 |
| 512 | 157 107 200 | 314 223 616 | 23 040 | 108 544 | 1 050 624 | 13 638 | 1 447 | 150 |
| 1024 | 517 113 856 | 1 034 246 144 | 51 200 | 237 568 | 4 198 400 | 20 200 | 2 177 | 123 |

## 5.6 Вартість в однакових одиницях

У таблиці 12 наведено дійсні флопи (строгий режим, адаптивне правило, одиниці з підрозділу 4.7).

Кількість гейтів одного слова, поділена на $r^2$, становить 15,8, 56,7, 77,1, 74,9 і 61,6 при $r = 64, \ldots, 1024$. Ця величина не стала, тож дані не встановлюють квадратичного закону вартості. На перевірених ширинах слово дорожче навіть за пряме множення (рисунок 5). Це вартість окремого виконання слова; препринт так слово не використовує і переваги на цих ширинах не заявляє (підрозділ 4.7). Препринт посилається на Alman і Rao [3] з головним членом $(15/4)\, n \log_2 n$ дійсних операцій для DFT на степенях двійки; порівняно з цим еталоном розрив був би ще більшим.

**Компроміс між гейтами і глибиною при $r = 512$ (рисунок 6).** Строгий режим, три правила:

- адаптивне ($b = 4$): глибина 1 035 044, 19 638 528 гейтів;
- чанк розміру 1: глибина 2 446 116, 4 647 168 гейтів;
- cap2 ($b \leq 2$): глибина 2 059 044, 22 882 560 гейтів.

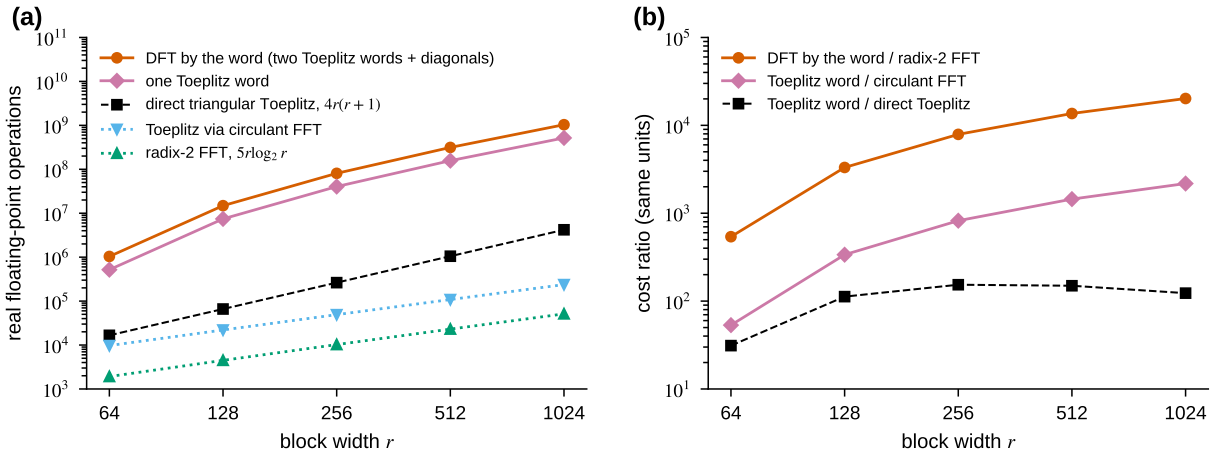



**Рис. 5:** (a) Дійсні флопи DFT через слово, одного Тепліцевого слова, прямого Тепліцевого множення, Тепліцевого множення через циркулянтний FFT і радікс-2 FFT. (b) Відношення вартостей у тих самих одиницях. Дані: таблиця 12.

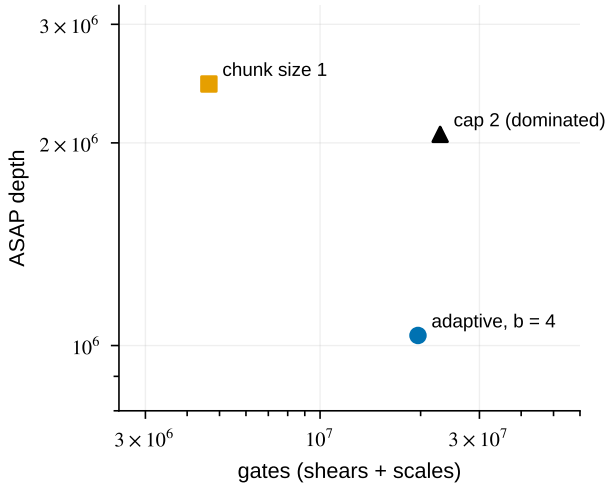



**Рис. 6:** Гейти і глибина при $r = 512$ для трьох правил у строгому режимі. Обидві осі логарифмічні. Розкиду по входах немає, бо топологія фіксована.

Адаптивне правило і чанк розміру 1 утворюють компроміс: у чанка розміру 1 у 4,2 раза менше гейтів і в 2,4 раза більша глибина. cap2 гірше за адаптивне правило за обома показниками, тобто серед цих трьох правил воно домінноване. Перевірено лише три правила. Інші розміри чанків, різні правила на різних рівнях рекурсії або інший розклад можуть дати кращі точки, тож ми не стверджуємо, що жодне правило не мінімізує обидві величини.

## 5.7 Раунди і конфлікти

У строгому режимі (`rounds_check.py`) виконання за раундами дає той самий результат, що й послідовне: різниця 0,0 при $r = 16, 32, 48, 64$. У межах раунду немає конфліктів координат: 0 при $r = 256$ (779 392 шари, 518 948 раундів, 1,50 шару на раунд) і при $r = 512$ (2 074 880 шарів, 1 035 044 раунди, 2,00 шару на раунд).

## 5.8 Час і пам'ять нашої реалізації

Один процес на ширину, float64, строгий режим, адаптивне правило, машина з підрозділу 4.9, команда `timing.py r`. Ми вимірюємо одне Тепліцеве слово на випадкових комплексних $h$ і $x$ (`numpy.default_rng(r)`, дійсні та уявні частини `standard_normal`, $h_0$ замінено

**Таблиця 13:** Python-реалізація (`timing.py`), вимірювання 2026-10-14 без паралельних кроків прогону. Побудова: підготовка скалярів і розкладу.

| $r$ | Гейтів | Побудова слова, с | Застосування до одного вектора, с | Пікова пам'ять, ГіБ | numpy.fft довжини $r$, мкс | Щільне Тепліцеве множення (numpy), мкс |
|---|---|---|---|---|---|---|
| 64 | 64 800 | 0,13 | 0,01 | 0,04 | 0,8 | 2,0 |
| 128 | 928 320 | 0,73 | 0,13 | 0,12 | 1,2 | 6,1 |
| 256 | 5 051 520 | 3,5 | 0,71 | 0,50 | 1,7 | 22,0 |
| 512 | 19 638 528 | 13,2 | 2,8 | 1,79 | 2,7 | 82,9 |

на $|h_0|+1$), а не повний ланцюжок DFT із двома словами і діагональними множниками. У коді RSS підписано як GB, але `ru_maxrss`/$1024^2$ на Linux дає ГіБ; у таблицях ужито ГіБ.

Вимірювання з таблиці 13 зроблено 2026-10-14 у прогоні `full-20261013-r3`. Прогін виконує кроки послідовно, тож кроки `timing-64`, ..., `timing-512` не перетиналися з іншими кроками прогону; кожен із них — окремий процес у чистому віртуальному середовищі (Python 3.12.3, numpy 1.26.4) з `OPENBLAS_NUM_THREADS` = `OMP_NUM_THREADS` = `MKL_NUM_THREADS` = 1. Інших процесів на машині не контролювали. Чотири кроки разом тривали 24 с. Попереднє вимірювання тією самою командою, зроблене тоді, коли на машині працювали ще сім однопотокових обчислень (із 32 потоків), збережено без змін як запис тих умов у таблиці 17 (додаток A).

У цих прогонах вимірювання часу проміжні значення у float64 переповнюються при $r \geq 256$, тож виходи не використовуються як перевірка правильності; час вимірює виконання того самого фіксованого списку гейтів у Python, а точну правильність і числову поведінку перевірено окремими експериментами вище. Кількість гейтів однакова в обох вимірюваннях і дорівнює сумі зсувів і масштабувань адаптивного слова в таблиці 18. Відношення повторного часу до попереднього становить 0,81–0,88 для побудови, 0,81–1,00 для застосування і 0,73–0,81 для `numpy.fft`. Побудова слова в mp дорожча: 7–83 с при $r = 127$–$256$ з 250–450 знаками в попередніх журналах (таблиця 3) і 3–79 с у повторному прогоні. Ці числа описують нашу Python-реалізацію на цій машині, а не алгоритм препринту. Вони показують порядок величини: наша реалізація застосовує одне слово приблизно від $10^4$ ($r = 64$) до $10^6$ ($r = 512$) разів повільніше, ніж `numpy.fft` виконує все перетворення (і в повторному, і в попередньому вимірюванні), а пам'ять росте разом із кількістю гейтів. При $r = 512$ слово будується за 13,2 с (15 с у попередньому вимірюванні).

# 6 Обговорення

## 6.1 Показник і повна формула

З $\delta$ із (2) відношення головних множників дорівнює

$$\frac{n(\log n)^{\theta}(\log\log n)^{4-\theta}}{n\log n} = \frac{(\log\log n)^{3+\delta}}{(\log n)^{\delta}}. \tag{22}$$

Воно прямує до нуля, тож заявлена межа справді є $o(n\log n)$. Проте при практичних $n$ воно велике. При $n = 10^{300}$ з натуральними логарифмами ($\ln n \approx 690{,}8$, $\ln\ln n \approx 6{,}54$) відношення дорівнює $\approx 279{,}446$ без урахування прихованих сталих (з $\log_2$ було б $\approx 988$). Відношення стає меншим за 1 лише тоді, коли

$$\delta \ln\ln n > (3+\delta)\ln(\ln\ln n), \qquad \text{тобто при } \ln\ln n \approx 4{,}81494 \cdot 10^{14}. \tag{23}$$

Це не прогноз часу виконання, а пояснення, чому мала різниця показників не означає практичного прискорення. Числове значення $\theta$ перераховано в підрозділі 2.2; наші скінченні локальні експерименти не перевіряють заявленої глобальної асимптотичної межі часу, для якої потрібне аналітичне або формальне доведення.

## 6.2 Розрив між масштабами

$m = 10^6$ і $W_* = 2^{71}$ — параметри тензорної мережі, а не ширини локального компілятора. Локальні ширини в доведенні — перші $\ell$ непарних простих і один степінь двійки $2^e < 2r_{\ell+1}$, $\ell = \Theta(\log n / \log\log n)$ (`all-lengths.tex:13--56`); ширини цих типів (непарні прості, як-от 7, 31 і 127, і степені двійки) частково покрито нашими тестами. Конкретна мережна рекурсія застосовується лише в секторах з $k \geq K = m(71+1) = 72\,000\,000$ осями (`network.tex:494--498`), а $k \leq \ell$ (`synchronization.tex:82`). Тоді $\ell \geq 72\,000\,000$, і вже елементарна нижня межа $r_\ell \geq \ell + 2$ показує, що найбільша локальна ширина перевищує 1024 більш ніж у 70 тисяч разів. Це зіставлення найбільшої ширини, а не твердження, що всі осі великі. Екстраполяція нашої числової поведінки на такі ширини не обґрунтована.

## 6.3 Робота проти часу

Препринт обмежує повний час, включно з підготовкою скалярів і роботою з індексами. Ми рахуємо гейти слова та їхні флопи, а час і пам'ять вимірюємо лише для нашої реалізації (таблиці 13 і 17).

## 6.4 Оптимізації, залежні від значень

Пропуск нульових множників не змінює оператора, тож для числової реалізації оператора він допустимий. Але він виходить за межі конструкції препринту, бо потребує тесту на нуль комплексного коефіцієнта, якого конструкція не використовує (`local.tex:42`), і змінює об'єкт вимірювання: глибина і робота стають властивостями пари (ширина, коефіцієнти), а не ширини. Методологічний урок прямий: залежний від значень варіант може породжувати структурні ефекти, яких немає в досліджуваній конструкції. У нашому випадку він породив хибний перетин глибин між $r = 320$ і 336 та хибний розкид глибини між seed (підрозділ 3.6); усі наведені тут результати використовують строгу конструкцію. Вироджені входи ($h =$ ones) цікаві самі по собі, і їх слід аналізувати окремо, а не виключати; у строгому режимі вони дають ту саму структуру, що й загальні.

## 6.5 Числова нестійкість

Конструкцію препринту розраховано на точну арифметику, і стійкості препринт не заявляє. Наші дані показують, наскільки важливе це обмеження. У випадку Фур'є обумовленість Тепліцевого множника росте приблизно як $10^{cr}$, $c = \mathrm{Cl}_2(\pi/3)/(\pi \ln 10) \approx 0{,}1403$ (головний асимптотичний член скінченної обумовленості $D_H$), а повний ланцюжок втрачає 3–4 такі одиниці, тобто близько $0{,}5\,r$ десяткових знаків (142 при $r = 256$). У float64 ланцюжок непридатний уже з $r \approx 32$. Звичайний FFT стійкий. Пряме виконання цього слова (з природним порядком вузлів) у скінченній точності потребувало б на перевірених ширинах кількості знаків, що росте приблизно лінійно з $r$; це змінює бітову вартість. Препринт нічого не стверджує про бітову складність (`introduction.tex:101`), тож це не суперечність, а кількісна ілюстрація обмеження, яке він сам називає. Висновок стосується лише локальної складової: він не означає, що повний алгоритм нестійкий у будь-якій реалізації, і не встановлює нижньої межі вартості інших модифікацій.

### 6.6 Нижні межі

Межі Morgenstern і Ailon стосуються обмежених коефіцієнтів, унітарних гейтів або обмеженої обумовленості. Наші дані показують, що локальний компілятор справді лежить поза цими рамками: обумовленість його множників росте експоненційно з $r$. Це узгоджується з тим, що межі на нього не поширюються, але нічого не говорить про правильність мережі.

### 6.7 Що випливає для препринту

У межах перевіреного діапазону локальна складова поводиться так, як заявлено: реалізований оператор дорівнює $F_r(u)$ (доведено обчисленням у $\mathbb{Q}(\zeta_N)$ для восьми ширин 4–32; для інших перевірених $r$ — узгоджується з рівністю), топологія строгого слова залежить лише від $r$, а модель глибини сумісна з $O(\log^4 r)$. Розбіжностей із твердженнями препринту ми не знайшли. Це не підтверджує і не спростовує головної теореми, ефект якої походить від мережі. Для перевірки самого доведення препринту потрібен аудит мережі й синхронізації; звірку Lean-означень завершено в обсязі підрозділу 4.8. Наші вимірювання кількісно описують для локальної складової обмеження, які препринт називає сам (величезні сталі, точна арифметика; `introduction.tex:67`, `:101`), і узгоджуються з практичними зауваженнями Recht [11]; про правильність препринту вони нічого не говорять.

## 7 Обмеження і загрози валідності

**Обсяг перевірок.** Повний DFT доведено обчисленням у $\mathbb{Q}(\zeta_N)$ на восьми ширинах від 4 до 32; у $\mathbb{F}_p$ (необхідна умова) його перевірено для всієї матриці на десяти вибраних ширинах від 9 до 128, для кількох векторів при $r = 255, 256, 512$ і окремо в потоковому режимі при $r = 1024$ (ті самі два вектори у двох полях); у mp — при $r \le 256$. $C$-слоти доведено обчисленням у $\mathbb{Q}(\zeta_N)$ при $r = 4, 8, 16$, перевірено в $\mathbb{F}_p$ для всієї матриці при $r = 16, 31, 32, 33, 64$ і вибірково при $r = 128$, а в mp — при $r \le 256$. Тепліцеве слово з випадковими $h$ перевірено при $r \le 1024$; глибину виміряно при $r \le 1024$ і змодельовано до $r = 2^{17}$. У mp при $r = 96, 127, 256$ і в $\mathbb{F}_p$ при $r = 255, 256, 512$ перевірено кілька випадкових векторів, а не всю матрицю; у $\mathbb{F}_p$ при $r = 127$ перевірено всі базисні вектори.

**Сила доказу.** Точні перевірки в $\mathbb{F}_p$ — необхідна умова, а не доведення комплексної рівності; доведеннями обчисленням є лише перевірки в $\mathbb{Q}(\zeta_N)$. Збіг у mp — лише числова узгодженість.

**Глибина.** ASAP-глибина — конкретний розклад, а не оптимум і не нижня межа. Еквівалентність нашого правила чанка правилу препринту встановлено лише для $b \le 1024$ (підрозділ 3.3). Модель глибини використовує емпіричні $g(b)$ і $L(b)$ і йде лише гілкою $\lfloor v/2 \rfloor$.

**Аналіз нестійкості.** Вимірювання по етапах описують спостережений розподіл втрати знаків (таблиця 7); розклад росту описує прирости рекордного максимуму по рівнях рекурсії і фазах (таблиця 14); а контроль точності фаз (таблиця 15, $r = 64, 128$) вимірює чутливість похибки до точності міжблокових чанків для перевірених входів. Разом ці діагностики не дають повного причинного розкладу похибки округлення. Підготовки коефіцієнтів цей контроль не охоплює, і для $T$ після нього залишається значна втрата. Не зроблено: розкладу до рівня окремих чанків, контролю для інших розподілів коефіцієнтів із можливим взаємним скороченням і ширин понад $r = 128$.

**Вартість.** Підготовку скалярів, індекси і пам'ять виміряно лише для нашої Python-реалізації; флопи пораховано за стандартною схемою без спеціальних коефіцієнтів.

**Lean.** Ми повторно перевірили замикання головної цілі, аксіоми і відповідність означень `DFTGoal`/`ConvGoal`/`RAM` формулюванням препринту. Формалізація доводить потрібну межу через іншу мережу; `network.tex`, $\Delta$ і `net:tensor-bound` нею не перевіряються. Тіл доведень ми не перечитували; відповідність `Bill` фізичній word-RAM залишається модельним судженням; окремої теореми симуляції немає. Це відтворення чужої механічної перевірки, а не наше власне незалежне доведення.

**Головна теорема.** Числове значення $\theta$ перераховано (підрозділ 2.2), але головний асимптотичний виграш, мережу, синхронізацію, алгоритм для довжин усіх розмірів і синтез локальними експериментами не перевірено.

**Загрози валідності.** Точні перевірки виконує `generic_word.py`, тоді як чисельні використовують `mp_exact`/`mp_compiler` і `conv_dag`/`rounds`. Це окремі реалізації однієї описаної конструкції, тож рівність у точному полі безпосередньо стосується саме точного перевірника. Додаткове порівняння послідовностей шарів і пар координат (без коефіцієнтів) показало, що три строгі конструктори збігаються при $r = 4, 7, 8, 9, 12, 15, 16, 31, 32, 33, 64, 128$ для $h_k = 1/(k+1)$; це підтримує структурну відповідність, але не доводить збігу їхніх коефіцієнтів і не є твердженням для всіх ширин. Власний розклад зсуву, рання зупинка правила чанка і ASAP-розклад — рішення реалізації; їхні перевірені межі наведено вище. Фазові діагностики і діагностики випадкового Тепліцевого слова використовують один seed на ширину; ширшого статистичного узагальнення не заявлено. Час залежить від машини і від одночасного навантаження.

# 8 Доступність даних і коду

Код, дані і сирі журнали повного прогону `full-20261013-r3` разом із його маніфестом і хешами доступні за адресою `https://github.com/cyberdid/fft-power-saving-verification` за ліцензією Apache License 2.0. Шляхи нижче задано відносно кореня цього репозиторію.

- `release/README_REPRODUCE.md`: кроки відтворення і відповідність скриптів таблицям.
- `release/run_all.sh quick|full`: ізольований прогін, тобто копія джерел, потім сирі результати, потім CSV-файли цього прогону, потім звірка з ними всіх таблиць рукопису, потім рисунки з тих самих CSV-файлів. Кожен прогін має власний каталог і маніфест команд, кодів завершення і хешів. Режим `full` виконує всі 135 кроків; режим `quick` явно пропускає шість найдовших експериментів і тому не підтверджує повного відтворення. Перерваний прогін можна продовжити в тому самому середовищі; при продовженні перевіряється, що зафіксовані джерела, середовище і результати завершених кроків не змінилися.
- `release/evidence/full-20261013-r3/`: сирі журнали, агреговані дані, рисунки, freeze середовища і маніфест повного прогону, використаного в цій статті; маніфест містить SHA-256 кожного зафіксованого файлу джерел, кожного журналу і кожного виходу. Файли побайтово ідентичні файлам прогону і тому зберігають абсолютні шляхи машини, на якій його виконано. Криві порівняння для варіанта з пропуском нулів на рисунку 3 архівні і позначені відповідно.
- `paper/data/supplemental/fp1024/`: дані, журнали та інструкції з відтворення окремого потокового $\mathbb{F}_p$-тесту при $r = 1024$ (`paper/scripts/stream_exact_dft.py`), який не входить до 135 кроків.

- `release/requirements-lock.txt`: зафіксовані прямі й транзитивні залежності, встановлені в чисте середовище з успішним `pip check`; `release/requirements-full-freeze.txt`: повний freeze середовища розробки (запис, а не lock-файл); `release/ENVIRONMENT.txt`.
- `release/lean/`: `lean-toolchain`, `lakefile.lean`, `lake-manifest.json`, `AxiomCheck.lean`, `MODEL_AUDIT.md` і журнали збірки.
- `release/SHA256SUMS`: контрольні суми коду, даних і рисунків цієї версії; вони засвідчують цілісність зафіксованих файлів, а не доведення будь-якої теореми.

Код міститься в `allength/` (серед іншого `conv_dag.py` з прапорцем `KEEP_ZERO_MULTIPLIERS`, `mp_exact.py` з `ROOT_SIGN`, `exact_field.py`, `generic_word.py`, `rounds.py`, `mp_compiler.py`, `compiler.py`, `depth_recurrence.py`); скрипти — у `paper/scripts/`, агреговані дані — у `paper/data/*.csv`. Препринт доступний у репозиторії `https://github.com/openai/math` (коміт каталогу 301488868, HEAD fd4aeeb).

# 9 Висновок

Ми незалежно перевірили локальний компілятор препринту OpenAI у строгій формі, де топологія залежить лише від ширини, і підтвердили цю властивість аудитом коду і хешами структури. Для восьми ширин від 4 до 32 ми довели обчисленням у циклотомічних полях, що матриця точного конструктора дорівнює $F_r(u)$. Для вибраних ширин до $r = 512$ і в окремому потоковому тесті при $r = 1024$ точні перевірки в скінченних полях дають необхідну умову (вибірково при $r = 255, 256, 512, 1024$), а арифметика високої точності до $r = 256$ узгоджується з рівністю. Шлях через $C$-слоти (наш розклад зсуву; розщеплення і спряження через DAG з оберненими коренями, як у препринті) доведено обчисленням при $r = 4, 8, 16$.

У скінченній точності ця точна конструкція сильно нестійка; препринт стійкості не заявляє, тож це не суперечить жодному його твердженню. Обумовленість Тепліцевого множника у випадку Фур'є росте приблизно як $10^{cr}$, $c \approx 0{,}1403$ (головний асимптотичний член скінченної обумовленості, через функцію Клаузена); повний ланцюжок втрачає близько $0{,}5\,r$ десяткових знаків, майже всі — у двох Тепліцевих словах, і у float64 непридатний уже з $r \approx 32$, хоча $\kappa_2(F_r) = 1$. Виміряна глибина строгого слова до $r = 1024$ і модель глибини до $2^{17}$ сумісні з верхньою межею $O(\log^4 r)$. Вартість окремого виконання слова в дійсних флопах на перевірених ширинах на два–чотири порядки перевищує FFT і перевищує навіть пряме множення; препринт так слово не використовує (підрозділ 4.7). Варіант, що відкидає точно нульові множники, залежить від значень і породжує ефекти глибини, відсутні в строгій конструкції; тому всі результати тут отримано для останньої.

Наші локальні експерименти не перевіряють головного асимптотичного виграшу з $\theta$. Перекомпільована нами Lean-формалізація OpenAI доводить `DFTGoal` з потрібною верхньою межею через іншу мережу; конкретна `network.tex` та її $\Delta$ нею не перевіряються. Звірку означень завершено в обсязі підрозділу 4.8; адекватність `Bill` щодо фізичної RAM залишається модельним судженням, а тіл доведень ми построково не перечитували. Наші результати не спростовують теореми і не є незалежним доведенням її головного ефекту; вони окреслюють перевірену локальну реалізацію та межі її використання у скінченній точності.

# Література

[1] Nir Ailon. A lower bound for Fourier transform computation in a linear model over $2 \times 2$ unitary gates using matrix entropy. arXiv:1305.4745, 2013.

[2] Nir Ailon. An $n \log n$ lower bound for Fourier transform computation in the well conditioned model. arXiv:1403.1307, 2014. v1 2014-03-06; title of the current version v5, 2014-07-24. The preprint's bibliography cites the earlier title "An $\Omega((n \log n)/R)$ lower bound for Fourier transform computation in the $R$-well conditioned model".

[3] Josh Alman and Kevin Rao. Faster Walsh–Hadamard and discrete Fourier transforms from matrix non-rigidity. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing (STOC 2023)*, pages 455–462. ACM, 2023. doi: 10.1145/3564246.3585188. Extended version: arXiv:2211.06459, 2022-11-11.

[4] James R. Bunch. Stability of methods for solving Toeplitz systems of equations. *SIAM Journal on Scientific and Statistical Computing*, 6(2):349–364, 1985. doi: 10.1137/0906025.

[5] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965. doi: 10.1090/S0025-5718-1965-0178586-1.

[6] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, Philadelphia, second edition, 2002. doi: 10.1137/1.9780898718027.

[7] Thomas Kailath, Sun-Yuan Kung, and Martin Morf. Displacement ranks of a matrix. *Bulletin of the American Mathematical Society (New Series)*, 1(5):769–773, 1979. doi: 10.1090/S0273-0979-1979-14659-7.

[8] Jacques Morgenstern. Note on a lower bound on the linear complexity of the fast Fourier transform. *Journal of the ACM*, 20(2):305–306, 1973. doi: 10.1145/321752.321761.

[9] OpenAI. An explicit power saving for the exact discrete Fourier transform. Preprint, 2026-09-25, OpenAI Math repository, `https://github.com/openai/math`, September 2026. Directory `preprints/An-explicit-power-saving-for-the-exact-discrete-Fourier-transform-September-25-2026` (`main.pdf`; sources `build/main.tex`, `build/sections/*.tex`, `build/references.bib`). Last change of the directory: commit 301488868, 2026-10-07; repository HEAD fd4aeeb. The Lean formalisation `lean/OAI/Computability/FourierTransform` (scope described in `lean/docs/130.md`) is a separate artifact of the same repository.

[10] OpenAI. Finite tensor savings and exact Fourier circuits. Preprint, 2026-09-25, OpenAI Math repository, `https://github.com/openai/math`, September 2026. Directory `preprints/Finite-tensor-savings-and-exact-Fourier-circuits-September-25-2026`.

[11] Benjamin Recht. Faster Fourier transforms. argmin (blog), 2026-10-12, `https://www.argmin.net/p/faster-fourier-transforms`, 2026. Not peer reviewed; cited as context for the critique.

[12] Lothar Reichel. Newton interpolation at Leja points. *BIT Numerical Mathematics*, 30(2): 332–346, 1990. doi: 10.1007/BF02017352.

# A Додаткові таблиці

**Таблиця 14:** Розклад росту проміжних величин по рівнях і фазах (одна випадкова $x$, seed $= r$; ріст у $\log_{10}$). В останньому стовпці наведено прирости за шириною блоку $v$.

| $r$ | Етап | Ріст | У фазі чанків | У прямому слові | Розподіл по рівнях $v$ (приріст) |
|---|---|---|---|---|---|
| 32 | $T^{\mathsf{T}}$ | 6,5 | 100% | 0% | $v$=32: 3,6; $v$=16: 2,9 |
| 32 | $T$ | 3,0 | 50% | 50% | $v$=16: 1,5; $v$=8: 1,5 |
| 64 | $T^{\mathsf{T}}$ | 19,1 | 100% | 0% | $v$=64: 9,4; $v$=32: 6,3; $v$=16: 3,4 |
| 64 | $T$ | 9,2 | 67% | 33% | $v$=16: 5,1; $v$=8: 3,1; $v$=32: 1,0 |
| 128 | $T^{\mathsf{T}}$ | 43,9 | 100% | 0% | $v$=128: 28,9; $v$=32: 9,5; $v$=64: 5,4 |
| 128 | $T$ | 21,7 | 84% | 16% | $v$=16: 9,4; $v$=32: 5,8; $v$=8: 3,5; $v$=64: 2,8; $v$=128: 0,3 |

**Таблиця 15:** Втрачено знаків при DPS, коли вибрана фаза виконується на еталонній точності, а решта — на DPS; у дужках — зміна відносно «усе на DPS».

| $r$ | Етап | DPS | Усе на DPS | Еталонна фаза чанків | Еталонна пряма фаза |
|---|---|---|---|---|---|
| 64 | $T^{\mathsf{T}}$ | 50 | 17,8 | 3,3 (−14,5) | 17,8 (0,0) |
| 64 | $T$ | 50 | 21,9 | 11,6 (−10,3) | 21,9 (0,0) |
| 128 | $T^{\mathsf{T}}$ | 90 | 42,6 | 5,2 (−37,5) | 42,6 (0,0) |
| 128 | $T$ | 90 | 50,6 | 22,6 (−27,9) | 51,0 (+0,4) |

**Таблиця 16:** Модель строгої глибини на степенях двійки (значення моделі, а не вимірювання).

| $r$ | $b$ | $d_{\text{model}}(r)$ | $d_{\text{model}}/\log_2^4 r$ |
|---|---|---|---|
| $2^{12}$ | 16 | 7 383 844 | 356,1 |
| $2^{13}$ | 32 | 11 021 092 | 385,9 |
| $2^{14}$ | 64 | 15 182 628 | 395,2 |
| $2^{15}$ | 128 | 19 868 452 | 392,5 |
| $2^{16}$ | 256 | 25 078 564 | 382,7 |
| $2^{17}$ | 512 | 30 812 964 | 368,9 |

**Таблиця 17:** Попереднє вимірювання часу (`timing.py`), зроблене тоді, коли на машині працювали ще сім однопотокових обчислень. Стовпці — як у таблиці 13.

| $r$ | Гейтів | Побуд., с | Застос., с | Пам., ГіБ | fft, мкс | Щільне, мкс |
|---|---|---|---|---|---|---|
| 64 | 64 800 | 0,16 | 0,01 | 0,04 | 1,1 | 4,0 |
| 128 | 928 320 | 0,88 | 0,16 | 0,12 | 1,5 | 14,0 |
| 256 | 5 051 520 | 4,3 | 0,88 | 0,50 | 2,1 | 57,1 |
| 512 | 19 638 528 | 15,0 | 3,1 | 1,78 | 3,5 | 222,4 |

**Таблиця 18:** Аудит топології строгого слова: 12 ширин × 2 правила × 3 входи (seed = $r$, інший seed, $h$ = ones). Останній стовпець показує, чи однаковий хеш структури для трьох входів.

| $r$ | Правило | Глибина | Шарів | Гейтів | Однаковий хеш |
|---|---|---|---|---|---|
| 64 | адаптивне | 37668 | 50464 | 64 800 | так (3 входи) |
| 64 | чанк 1 | 37668 | 50464 | 64 800 | так (3 входи) |
| 128 | адаптивне | 133924 | 197184 | 928 320 | так (3 входи) |
| 128 | чанк 1 | 152356 | 215616 | 277 056 | так (3 входи) |
| 256 | адаптивне | 518948 | 779392 | 5 051 520 | так (3 входи) |
| 256 | чанк 1 | 611108 | 889984 | 1 143 936 | так (3 входи) |
| 288 | адаптивне | 656793 | 986400 | 6 393 312 | так (3 входи) |
| 288 | чанк 1 | 773433 | 1126368 | 1 447 776 | так (3 входи) |
| 320 | адаптивне | 811515 | 1239360 | 7 920 960 | так (3 входи) |
| 320 | чанк 1 | 955515 | 1412160 | 1 815 360 | так (3 входи) |
| 336 | адаптивне | 626622 | 1098096 | 10 059 168 | так (3 входи) |
| 336 | чанк 1 | 1053510 | 1556736 | 2 001 216 | так (3 входи) |
| 352 | адаптивне | 497926 | 1015392 | 8 050 752 | так (3 входи) |
| 352 | чанк 1 | 1156166 | 1708480 | 2 196 288 | так (3 входи) |
| 384 | адаптивне | 592581 | 1208640 | 9 581 376 | так (3 входи) |
| 384 | чанк 1 | 1375941 | 2033472 | 2 614 080 | так (3 входи) |
| 512 | адаптивне | 1035044 | 2074880 | 19 638 528 | так (3 входи) |
| 512 | чанк 1 | 2446116 | 3614976 | 4 647 168 | так (3 входи) |
| 640 | адаптивне | 1617915 | 3285120 | 30 741 120 | так (3 входи) |
| 640 | чанк 1 | 3822715 | 5691520 | 7 317 120 | так (3 входи) |
| 768 | адаптивне | 1753797 | 3578496 | 40 617 600 | так (3 входи) |
| 768 | чанк 1 | 5504709 | 8195712 | 10 536 576 | так (3 входи) |
| 1024 | адаптивне | 1682212 | 4796928 | 64 639 488 | так (3 входи) |
| 1024 | чанк 1 | 9786148 | 14569984 | 18 731 520 | так (3 входи) |

# Б Доведення і виведення

## Б.1 Ненульовість коефіцієнтів розщеплення

*Доведення леми 2.* Оскільки $\kappa = 1 + |\mu|^2$ дійсне і $\kappa \geq 1$, маємо $t_1 = \kappa \neq 0$. За оберненою нерівністю трикутника $|t_2| = |\kappa - \mu| \geq |\kappa| - |\mu| = 1 + |\mu|^2 - |\mu|$. Виділяючи повний квадрат,

$$1 + |\mu|^2 - |\mu| = \Big(|\mu| - \tfrac{1}{2}\Big)^2 + \tfrac{3}{4} \;\geq\; \tfrac{3}{4} > 0.$$

Отже, $t_2 \neq 0$, і $t_1 + t_2 = \mu$ для кожного $\mu$, включно з $\mu = 0$. Міркування використовує те, що $\mu\bar{\mu} = |\mu|^2$ — невід'ємне дійсне число; тому воно не переноситься на $\mathbb{F}_p$, де ненульовість $t_1, t_2$ натомість перевіряється при кожному оберненні (підрозділ 4.2). □

## Б.2 Асимптотика Клаузена для $\log_{10}\kappa_2(D_H)$

Нагадаємо, що $\mathrm{Cl}_2(\varphi) = -\int_0^{\varphi} \log|2\sin(t/2)|\,dt$; її максимум на $[0, 2\pi]$ досягається при $\varphi = \pi/3$, $\mathrm{Cl}_2(\pi/3) \approx 1{,}01494$, і $\mathrm{Cl}_2(2\pi - \varphi) = -\mathrm{Cl}_2(\varphi)$.

**Твердження 2.** *Для $u = e^{2\pi i/r}$ і $D_H = \mathrm{diag}(H_0, \ldots, H_{r-1})$*

$$\log_{10}\kappa_2(D_H) = \frac{r}{\pi}\,\frac{\mathrm{Cl}_2(\pi/3)}{\ln 10} + o(r) \qquad (r \to \infty).$$

*Доведення.* Для діагональної матриці $\kappa_2(D_H) = \max_k|H_k| / \min_k|H_k|$. Оскільки $|1 - e^{2\pi i s/r}| = 2\sin(\pi s/r)$ при $0 < s < r$,

$$\ln|H_k| = \sum_{s=1}^{k} \ln\big(2\sin(\pi s/r)\big).$$

Доданок від'ємний при $\pi s/r < \pi/6$ і при $\pi s/r > 5\pi/6$ і додатний між ними; отже, $\ln|H_k|$ мінімальне при $k \approx r/6$ і максимальне при $k \approx 5r/6$. Наближаючи суму інтегралом (підінтегральна функція має на кінцях лише інтегровні логарифмічні особливості, тож похибка суми Рімана є $o(r)$) і підставляючи $t = x/2$, маємо

$$\ln|H_k| = \frac{r}{\pi}\int_0^{\pi k/r} \ln(2\sin t)\,dt + o(r) = -\frac{r}{2\pi}\,\mathrm{Cl}_2\Big(\frac{2\pi k}{r}\Big) + o(r)$$

рівномірно за $k$. При $2\pi k/r = 5\pi/3$ значення дорівнює $-\frac{r}{2\pi}\mathrm{Cl}_2(5\pi/3) = \frac{r}{2\pi}\mathrm{Cl}_2(\pi/3)$, а при $2\pi k/r = \pi/3$ — $-\frac{r}{2\pi}\mathrm{Cl}_2(\pi/3)$. Різниця дорівнює $\frac{r}{\pi}\mathrm{Cl}_2(\pi/3) + o(r)$; ділення на $\ln 10$ дає твердження з $c = \mathrm{Cl}_2(\pi/3)/(\pi\ln 10) = 0{,}14030576$. □

Скінченні значення в таблиці 6 — точні різниці часткових сум; вони відрізняються від $cr$ на $o(r)$, наприклад на $10^{-4}$ при $r = 1024$.

## Б.3 Локалізація для $\mathbb{F}_p$

*Доведення твердження 1.* Нехай $p = kN + 1$ — просте і $\omega \in \mathbb{F}_p$ має мультиплікативний порядок $N$. Оскільки $p \nmid N$, $\omega$ — корінь $\Phi_N$ за модулем $p$, і $\zeta_N \mapsto \omega$ задає сюр'єктивний гомоморфізм кілець $\phi : \mathbb{Z}[\zeta_N] \cong \mathbb{Z}[x]/(\Phi_N) \to \mathbb{F}_p$ з ядром $\mathfrak{p}$, яке є простим ідеалом. Нехай $R = \mathbb{Z}[\zeta_N]_{\mathfrak{p}} \subset \mathbb{Q}(\zeta_N)$ — локалізація, тобто дроби $a/b$ з $a, b \in \mathbb{Z}[\zeta_N]$ і $b \notin \mathfrak{p}$. Тоді $\phi$ однозначно продовжується до гомоморфізму кілець $\phi : R \to \mathbb{F}_p$, $\phi(a/b) = \phi(a)\phi(b)^{-1}$; раціональні сталі зі знаменниками, взаємно простими з $p$, лежать у $R$.

Індукцією вздовж нерозгалуженої програми покажемо, що кожне проміжне значення $z$ лежить у $R$ і що значення, обчислене в $\mathbb{F}_p$, дорівнює $\phi(z)$. Входи і сталі цю умову задовольняють. Кільцеві операції зберігають $R$ і комутують з $\phi$. Якщо $z \in R$ обертається і

**Таблиця 19:** Означення Lean проти препринту. $G$ = `Goal.lean`, $R$ = `RAM.lean`, $C$ = `FourierCircuit/Core.lean`.

| Означення Lean (файл) | Зміст | Висновок |
|---|---|---|
| `zeta`, `fourierMatrix` ($C$); `dft` ($G$) | $\zeta_n = \exp(2\pi i/n)$; $(\zeta_n^{jk})$; ненормоване $F_n x$ | Відповідає `introduction.tex` |
| Сталі $m$, $W_*$, $\Delta$, $\lambda$, $\theta$ ($G$) | Ті самі точні сталі, що й у (1) | Відповідає |
| `paperTime`, `decimalTime` ($G$) | $n(\ln n)^{\theta}(\ln\ln n)^{4-\theta}$; показник $1-10^{-13}$; натуральні логарифми | Відповідає `thm:main`, `cor:decimal` |
| `DFTProgram` ($G$) | Для всіх $n>0$: порядок кореня $0<D<1024\,n^3$ обчислюється й оплачується в моделі; `d.work` $+1+$ `result.work` $\le W(n)$; результат = `dft` $n\,x$ для всіх $x$ | Відповідає; порядок кореня обчислюється в моделі (сильніше) |
| `TimeBounds`, `DFTGoal` ($G$) | $W = O($`paperTime`$)$, $W = O($`decimalTime`$)$, $W = o(n\log n)$; фіксовані `order`, `solve`, $W$ | Відповідає |
| `ConvGoal` ($G$) | Білінійні добутки дозволено (`Prog true`); результат — лінійна згортка | Відповідає; сильніше (межа кореня, лише білінійні добутки) |
| `Paint`, `Bill`, атоми ($R$) | Окремі регістри скалярів і даних; кожен атом коштує 1; `inv` лише на скалярах з умовою ненульовості; немає комплексних сталих і атомів $\mathbb{C}\leftrightarrow\mathbb{N}$; розгалуження лише за цілими | Відповідає; теореми симуляції для word-RAM немає |
| `receipt` | $D =$ `receipt`$(n) < 8\,n^3$ у варіантах `_order` | Формула відрізняється від $D_*$ (`eq:master-root`); межу виконано |
| Внутрішня мережа (`lunch_big`, `alpha_theta`) | 32 двійкові координати на множник, 4060 трійок, межа $2^{50}$; $W = O\big(n(\ln n)^{\alpha}(\ln\ln n)^2\big)$, $\alpha = 1-2\cdot10^{-11} < \theta$ | Відрізняється від `network.tex`; $\Delta$ і `net:tensor-bound` не перевірено |
| Аксіоми (усі чотири головні теореми) | `propext`, `Classical.choice`, `Quot.sound` | Без `sorry`; 89 модулів у замиканні імпортів |

обчислення в $\mathbb{F}_p$ обертає $\phi(z)\neq 0$, то $z\notin \mathfrak{p}R$, отже, $z$ — оборотний елемент локального кільця $R$, $z^{-1}\in R$ і $\phi(z^{-1})=\phi(z)^{-1}$. Таким чином, вихід у $\mathbb{F}_p$ є образом виходу в $\mathbb{Q}(\zeta_N)$. Якщо останній дорівнює $F_r(\zeta_r)$, то перший дорівнює $F_r(\phi(\zeta_r))$; від супротивного, розбіжність у $\mathbb{F}_p$ означає розбіжність над $\mathbb{Q}(\zeta_N)$, тоді як збіг у $\mathbb{F}_p$ не виключає ненульової різниці, що лежить у $\mathfrak{p}R$.

Комплексне спряження на $\mathbb{Q}(\zeta_N)$ — це автоморфізм $\sigma:\zeta_N\mapsto\zeta_N^{-1}$. DAG, обчислений з оберненими коренями, обчислює $\sigma(\mu)=\bar{\mu}$ над $\mathbb{Q}(\zeta_N)$, а той самий DAG, обчислений у $\mathbb{F}_p$ з $\omega^{-1}$ замість $\omega$, за наведеним міркуванням обчислює $\phi(\sigma(\mu))$. Отже, перевірка шляху через $C$-слоти в $\mathbb{F}_p$ перевіряє образ того самого спряженого виразу, хоча саме поле $\mathbb{F}_p$ не має нетривіального автоморфізму. □

# B Підсумок аудиту формулювання в Lean

У таблиці 19 підсумовано звірку означень Lean з препринтом (підрозділ 4.8). Тіл доведень не перечитували; їхню коректність для цих означень перевіряє ядро Lean.